Swarm and Evolutionary Computation 98 (2025) 102058

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

An offline data-driven process for learning operator selection from metaheuristic search traces

Panagiotis Kalatzantonakis, Angelo Sifaleras *, Nikolaos Samaras

Department of Applied Computer Science, University of Macedonia, 156 Egnatia Str., Thessaloniki 54636, Greece

ARTICLE INFO

Keywords:
Learning-guided search
Adaptive operator selection
Variable neighborhood search
Vehicle routing problem
Machine learning for optimization
Offline trained models
Edge-compatible deployment

ABSTRACT

Trained Reward-based Action Classification Engine (TRACE) is a general process for capturing operator outcome data during metaheuristic search, training classifiers to predict whether an operator will yield an improved solution, and deploying those models to guide neighborhood selection during future search runs. This study introduces TRACE-VNS, a modular extension of General Variable Neighborhood Search (GVNS) applied to the Capacitated Vehicle Routing Problem (CVRP), where neighborhood selection is driven by these offline-trained models. Classifiers are trained on features extracted from GVNS traces, including action history, graph metrics, temporal state, and Upper Confidence Bound (UCB) indicators. Twelve classifiers, including tree ensembles, neural networks, and kernel-based models, are benchmarked using the Precision–Recall Area Under the Curve (PR-AUC) to evaluate predictive quality. Empirical results show that TRACE-VNS improves convergence speed and final solution quality over conventional GVNS across 84 CVRP instances. A detailed feature importance analysis identifies strong contributors, offering insights into the effective selection of operators. TRACE requires no runtime exploration or feedback loops and can generalize to other metaheuristics through minimal structural adaptation.

1. Introduction

1.1. Foundations of CVRP and metaheuristic approaches

1.1.1. CVRP: Problem statement and significance CVRP problem statement

The CVRP, first introduced by Dantzig and Ramser [1], poses a complex NP-hard optimization challenge central to logistics, distribution, and e-commerce. This problem involves optimizing the routes of a fleet of vehicles, each with a fixed capacity, to meet the demands of various customers efficiently. The fundamental mathematical framework of CVRP, set forth by Laporte [2] and refined by Baldacci et al. [3], delineates the constraints and objectives integral to solving this problem. For an in-depth exploration of the mathematical model, refer to Appendix A. The combinatorial nature of CVRP results in exponential growth in the solution space, where traditional exact methods become computationally infeasible. Heuristic methods, while effective, often fail to scale efficiently or adapt to dynamic problem instances. Machine learning offers a data-driven approach to refine neighborhood exploration and improve the consistency of the solution [4–8].

1.1.2. Metaheuristics for combinatorial optimization

Metaheuristics offer an advantage in CVRP by enabling scalable exploration of complex solution spaces. VNS, in particular, benefits from ML-guided neighborhood selection due to its modular structure and dynamic search mechanism.

The incorporation of machine learning into metaheuristics marks a significant advancement, reflecting a shift toward more adaptive and intelligent optimization frameworks. Unlike traditional randomized or fixed-parameter heuristics, ML-based metaheuristics allow dynamic adaptation to problem structures, improving convergence and solution quality.

Our research is based on the GVNS framework, a well-established metaheuristic for combinatorial optimization. By integrating ML, we set new benchmarks for solving vehicle routing problems. Beyond improving GVNS efficiency, a central objective of this study is to identify generalizable feature classes that predict operator effectiveness in structurally distinct instances. By systematically ranking feature contributions across diverse models, we provide insights that can inform future ML-augmented metaheuristics.

1.1.3. General variable neighborhood search: Principles and applications

The Variable Neighborhood Search (VNS) is a well-known metaheuristic for solving complex problems like CVRP. Originally from the work of Mladenović et al. [9], VNS is known for its adaptability to

* Corresponding author.
E-mail addresses: pkalatzantonakis@uom.edu.gr (P. Kalatzantonakis), sifalera@uom.gr (A. Sifaleras), samaras@uom.edu.gr (N. Samaras).

https://doi.org/10.1016/j.swevo.2025.102058
Received 10 May 2025; Received in revised form 23 June 2025; Accepted 25 June 2025
Available online 19 July 2025
2210-6502/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
VNS limitations and mitigation strategies from literature.

VNS limitation	Mitigation strategies	Bibliographic sources
High computational complexity	- **Parallel computing**: Distributed processing to manage large-scale problem sizes.	[11–15]
	- **Reduced neighborhood size**: Limiting neighborhoods considered in iterations to lower computational cost.	[9,16,17]
	- **Early stopping**: Termination rules to prevent unnecessary iterations.	[8,17]
Local optima entrapment	- **Diverse neighborhoods**: Using a wide range of structures to escape local optima.	[9,16–18]
	- **Hybrid algorithms**: Combining VNS with other metaheuristics (e.g., Genetic Algorithms, Particle Swarm Optimization, Simulated Annealing, Tabu Search).	[19–27]
Parameter adjustment and neighborhood design	- **Adaptive learning**: ML-based dynamic adjustment of parameters, neighborhood strategies, and search behavior.	[28–34]
	- **AI/ML and RL techniques**: Using AI/RL to identify and optimize neighborhoods.	[8,30,35–38]

exploring various neighborhoods. In particular, it represents over 20% of the metaheuristic literature, highlighting its widespread acceptance and effectiveness [10].

The strength of VNS lies in its systematic exploration of neighborhood structures, enabling adaptive search behavior. This study enhances GVNS by using ML-based predictions to dynamically select the most promising neighborhood at each step, balancing exploration and exploitation. This fusion improves solution quality by promoting faster convergence and more effective escape from local optima. For a detailed explanation of GVNS and its integration with ML, refer to Appendix A: GVNS.

1.2. Limitations of traditional VNS and recent innovations

VNS is a cornerstone metaheuristic in tackling complex optimization tasks, such as CVRP, yet it grapples with challenges such as entrapment in local optima, computational intensity, and strategic selection of neighborhoods. These challenges underscore the need for a more adaptive search strategy.

ML offers a solution by improving the neighborhood exploration process and improving the efficiency of decision making. Table 1 summarizes key challenges in VNS and the corresponding mitigation strategies from the literature.

1.3. Rationale for ML-augmented neighborhood selection

GVNS is a well-established metaheuristic for solving complex combinatorial problems such as CVRP. However, its performance is highly dependent on the selection of neighborhood operators, which is typically governed by static or randomized strategies that are not responsive to the evolving search state.

In contrast to online learning approaches that adapt policies during execution, TRACE employs a supervised model trained offline on historical operator outcomes. This design supports deterministic, low-latency inference without runtime reward tracking, making it well-suited for deployment in embedded, IoT, or edge computing environments where reproducibility, resource constraints, or regulatory constraints preclude adaptive exploration.

Supervised ML provides a data-driven alternative by enabling pretrained models to predict operator effectiveness based on structural features and historical outcomes. Unlike reinforcement learning (RL), which requires continuous feedback and exploration trade-offs, supervised ML supports immediate, low-overhead inference and benefits from offline training stability. This improves convergence rates, reduces variance in solution quality, and reveals interpretable feature patterns relevant to algorithm design. Furthermore, the use of supervised learning facilitates the isolation and systematic evaluation of feature signals, prioritizing scientific insight into which temporal, structural, and reinforcement-based variables generalize best across problem classes.

Recent studies have shown that ML can improve metaheuristics by guiding search dynamics, exploiting the structure of instances, and accelerating optimization [7,33,34,39–41]. In the context of CVRP, ML has been used to reduce the solution space [42], control neighborhood transitions using neural models and RL policies [43], and extract spatial features related to solution quality [44]. Furthermore, random forest-based operator guidance has proven effective in multi-objective evolutionary settings [45].

Building on this foundation and previous RL-based GVNS efforts [8], we propose a supervised ML-guided framework in which operator selection is driven by models trained on outcome history, graph structure, and search progress. Compared to RL-driven metaheuristics, which typically require online reward collection, exploration–exploitation balancing, and longer convergence times, the supervised approach used in TRACE-VNS enables faster, stable deployment with minimal runtime overhead. Although RL offers adaptivity during search, our results indicate that offline-trained supervised models achieve strong generalization with lower computational cost, making them preferable for fixed-horizon or batch optimization scenarios. This shifts the paradigm from stochastic heuristics to informed, modular control, promoting generalizability and interpretability.

Table 2 summarizes representative ML strategies that improve VNS behavior in the literature.

Supervised ML thus enables GVNS to evolve from fixed heuristics toward adaptive, prediction-informed optimization. The next sections present the proposed framework, evaluation methodology, and empirical results.

1.4. Contributions and research objectives

This study introduces a supervised learning framework that enhances neighborhood selection in GVNS using Machine Learning. The main contributions are as follows.

1. **Offline-Trained ML-Guided Neighborhood Selection:** We embed offline-trained classifiers into GVNS to dynamically guide operator selection, achieving up to 31% improvement in solution quality and 30% faster convergence compared to standard GVNS.
2. **Systematic Evaluation of Structural, Temporal, and Reinforcement Features:** Feature relevance is assessed using model-based (SHAP, MDI, TreeInterpreter), statistical (Chi-Squared, Pearson, LASSO) and dimensionality reduction techniques (PCA, autoencoders, intrinsic dimension estimation), covering structural, temporal, and UCB-based signals.
3. **Feature Trends Favoring Temporal and Reinforcement Signals:** Temporal and reinforcement-based features outperform engineered graph metrics in predictive trends, providing insights for feature prioritization in ML-enhanced metaheuristics.
4. **Robust Generalization Across Structurally Distinct Instances:** Models trained on A-Set instances generalize effectively to structurally different B-Set and X-Set problems without retraining, validating the robustness and transferability of the predictive selection strategy.

Table 2
Solution-based strategies for enhancing VNS performance.

Technique	Description
Offline adaptive strategy	Uses pre-trained models to guide operator selection, construct neighborhoods, or tune search parameters based on historical patterns [46–48].
Online adaptive strategy	Updates the search process in real time using state features through RL or feedback-driven heuristics [6,8,49].
Instance-specific heuristic adaptation	Applies handcrafted or learned rules tailored to individual instances; effective but typically non-generic [50].
Adaptive neighborhood design	Modifies neighborhood structures dynamically using ML, statistical inference, or metaheuristic hybrids [49,51–54].

Table 3
Local search operators in GVNS for symmetric CVRP.

Acronym	Local operator	Operation scope	Description
RL	Relocate	Intra-route	Moves a customer to another position within the same route.
SWAP	Swap	Intra-route	Exchanges two customers within the same route.
2OPT	2-Opt	Intra-route	Reverses the sequence of nodes between two selected edges.
3OPT	3-Opt	Intra-route	Reorganizes three edges in a route to optimize the path.
SH22	Shift 2-2	Inter-route	Swaps two customers between different routes.
SH21	Shift 2-1	Inter-route	Moves two customers from one route in exchange for one from another.
CRJN	Cross Join	Inter-route	Swaps sub-routes between different routes.
R2OP	Relocate-2opt	Hybrid	Moves a customer to a new route followed by 2-opt in the modified route.

5. **Modular Pipeline for ML Integration into Metaheuristics:** The modular architecture separates feature extraction, model inference, and search execution, allowing the proposed approach to be adaptable in principle to other metaheuristic frameworks, such as Simulated Annealing or Genetic Algorithms, pending suitable adjustments to their operator interfaces and state representations.

Overall, this work demonstrates how supervised learning can enhance metaheuristic control through informed, modular, and transferable operator selection, laying the groundwork for future extensions involving dynamic adaptation and cross-domain optimization.

2. Background

2.1. Benchmark instances used for training and evaluation

We tested our approach on 84 CVRP instances with 32 to 256 customers [55]. We selected instances from sets A, B [56], X [57], and E [58], covering 5 to 73 vehicles. This selection ensures balanced testing and aligns with the guidelines in [8] for fair algorithm comparison.

2.2. Local search moves in GVNS

A critical aspect of our GVNS approach is the strategic use of various neighborhood operators, which are instrumental in defining the local search space and dynamics. Each operator has a specific role in manipulating the solution structure, thereby enabling thorough exploration and exploitation of the search landscape. Table 3 lists and describes each neighborhood operator used in our study. This detailed view helps in understanding their individual contributions and interactions within the GVNS framework.

2.3. Choosing between offline and online operator guidance

Offline-trained models provide a modular, inference-efficient mechanism for guiding neighborhood selection in metaheuristics. Unlike online adaptation schemes that rely on exploration–exploitation tradeoffs or reward feedback, offline models leverage historical search traces to predict operator effectiveness based on structural and temporal features. This subsection summarizes the operational and deployment considerations that inform the choice between offline and online integration strategies.

Offline models are suitable when the instance distribution is relatively stable and representative traces are available. They are favored in applications with reproducibility requirements, edge deployment constraints, or strict latency budgets. The absence of runtime exploration ensures consistent behavior and avoids performance volatility during early-stage search. In contrast, online learning methods (e.g., bandit-based schemes) are preferable in settings with high instance variability or when adaptivity to unseen configurations is critical. Offline approaches also support traceability, modular testing, and decoupled development pipelines. The trained model becomes a reusable asset deployable across runs or systems, making it ideal for operational environments with limited tolerance for in-situ learning. However, they may underperform in domains with rapid distributional drift or where runtime feedback is indispensable (see Table 4).

Overall, the decision to adopt offline or online guidance depends on the balance between stability, adaptivity, and operational constraints. TRACE, in its current design, emphasizes offline integration for its low overhead, reproducibility, and alignment with edge or embedded deployment scenarios, where predictive operator selection must be deterministic and computationally lightweight.

3. Methodology

In this section, we describe the methodology used in our study, including the dataset and problem instances used, the preprocessing steps, the integration of various supervised machine learning algorithms used to identify optimal neighborhoods within VNS, and the experimental framework constructed to evaluate the efficacy and performance of these predictive models.

3.1. Offline dataset construction and analysis

3.1.1. Recording operator behavior with the GVNS pipeline

Training data were collected from **A-Set** CVRP instances using a modified GVNS framework (Fig. 1). To capture dynamic search behavior and minimize selection bias, we replaced the traditional VND flow with randomized neighborhood operator selection. This ensured diverse exploration patterns, critical for training robust ML models. At each iteration, we recorded Graph-based features, historical outcomes, and RL indicators to capture evolving solution structures and operator success patterns. A solution was considered improved if its total route cost strictly decreased compared to the previous iteration, without requiring a minimum threshold of improvement.

Table 4
Comparative overview: offline vs. online ML-guided operator selection.

Attribute	Offline pre-trained model	Online learning (e.g., UCB)
Inference cost	Near-zero; one-shot prediction	Continuous; reward-based updates
Data dependency	Requires historical traces	No prior data required
Runtime adaptation	Fixed; non-adaptive	Adaptive to instance and state
Deployment stability	Fully deterministic	Stochastic; run-to-run variance
Exploration risk	None; purely exploitative	Present; may yield poor early moves
Cold-start behavior	Strong from first iteration	May underperform initially
Edge deployment	Lightweight, self-contained	Requires state storage and logic
Suitability	Structured or repetitive instances	Highly variable or dynamic instances
Integration overhead	Minimal; no reward tracker	Requires reward feedback loop

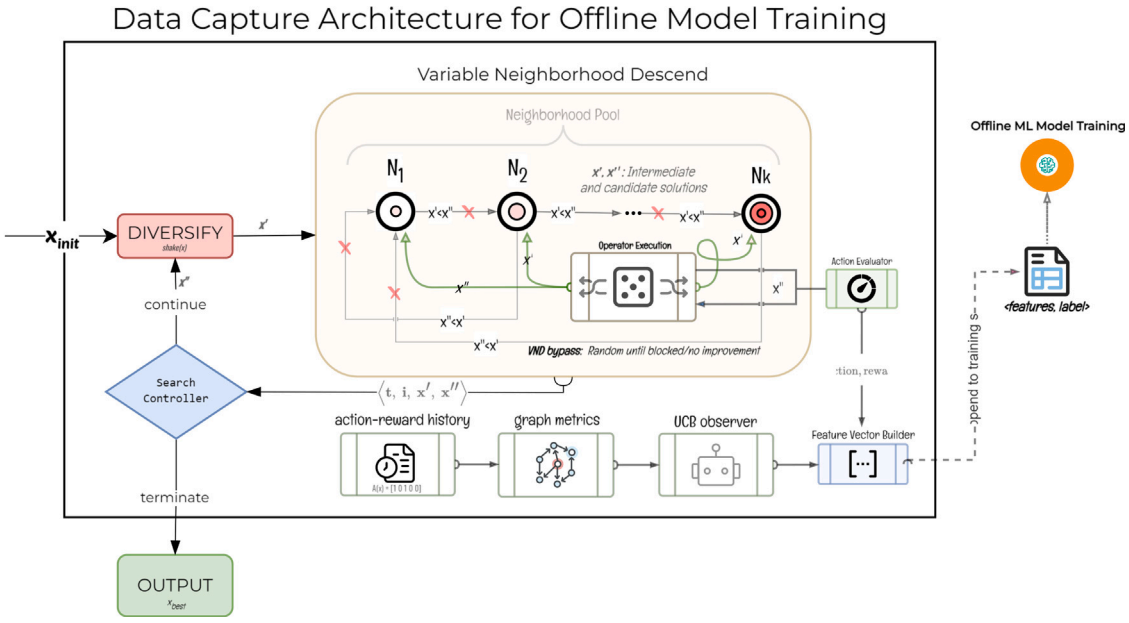


Fig. 1. Data Capture Architecture for Offline Model Training. During GVNS execution, operator outcomes and environment features are recorded to build a structured training set. Action-reward history, graph metrics, and UCB indicators are aggregated into feature vectors labeled by observed improvements in solution quality. These labeled vectors form the offline dataset used to train the ML model that guides operator selection in TRACE-VNS.

The resulting dataset was split into 80% for training and 20% for an unseen test set. Comparisons of ML models on this 20% subset are presented in Section 3.5.1. Subsequently, the best model is evaluated on the B-Set and X-Set, in completely unseen instances with different configurations, to assess its generalization capabilities.

This data collection module forms the foundation of our proposed pipeline, which we refer to as TRACE (Trained Reward-based Action Classification Engine). TRACE is a generalizable process for capturing search traces, labeling operator outcomes, and training predictive models to support operator selection in metaheuristics.

3.1.2. Data features collected

The extensive data captured during GVNS iterations, detailed in Table 5, spans 70 features across millions of rows and offers a multi-dimensional view of the factors influencing solution quality and neighborhood operator efficacy. Each feature category contributes uniquely to the ML model's predictive accuracy, informing the neighborhood operator selection process. For clarity, Appendix B.4 and Table B.19 summarize the abbreviations and notations used for non-graph features, including operator histories and UCB indicators.

3.1.3. Reinforcement learning indicators logging mechanism

UCB is a reinforcement learning strategy that balances exploration and exploitation by favoring actions with higher uncertainty and observed rewards.

The UCB observer, detailed in Appendix A (Appendix B.2.2 and Algorithm 3), passively logs which actions the UCB would have selected based on their computed values. For instance, if five actions are available, the UCB observer might output $[1, 0, 0, 1, 0]$, indicating that actions 1 and 4 had the highest UCB values. The UCB Selected (UCBs) column records this binary vector to support learning for neighborhood operator selection.

3.1.4. Instance specifics

Instance-specific features, including AFS and ClC, remain constant within a given problem instance but vary across different CVRP instances. These features allow the ML model to adjust its predictions based on the instance's complexity and scale, enhancing the model's adaptability to varying problem contexts.

3.1.5. Graph metrics

Graph metrics, as outlined in Table 6, play a pivotal role in understanding spatial relationships within optimized routes. These metrics, such as route compactness and intersections, enable the model to discern the structural characteristics of solutions, facilitating the identification of neighborhood operators that are most likely to yield effective solutions. These metrics contribute to a deeper understanding of the solution landscape, informing the ML model's predictive capabilities in selecting promising neighborhoods. Detailed formulas for each metric are provided in Appendix. GraphMetrics

3.1.6. Action history: Rolling window of operator outcomes

The rolling window captures the outcomes of the past five actions for each operator, enabling the ML model to recognize temporal dependencies and patterns. By incorporating recent outcome history, the

Table 5
Data features collected during GVNS iterations.

Source	Description	Feature format
Algorithmic features		
Algorithm state	Records the neighborhood operators chosen at each GVNS step, providing data to evaluate operator effectiveness over time.	Swap, 2-Opt, Relocate
Search process	Includes the current GVNS step and a rolling window of past action outcomes, helping the model identify temporal patterns in action performance.	Past 5 action outcomes (1, 0, 1, 0, 1)
Instance features		
Problem instance	Includes AFS and ClC to reflect the problem's complexity and scale. These remain fixed within an instance but vary across different CVRP cases.	AFS = 20, ClC = 150
Graph-based features		
Solution structure	Records spatial and structural details of the solution, such as route intersections and node-to-node distances (see Table 6 for details).	Avg route width = 3.2, Avg route depth = 0.68
Reinforcement learning features		
RL model	Records the UCB values for each action, showing which actions the UCB would have selected based on exploration–exploitation balance ('UCB Selected' column).	(1, 0, 0, 1, 0, 0, 0, 0)
Target outcome		
Target variable	Indicates whether the current solution improved.	1 = improved, 0 = not improved

AFS = Active Fleet Size, ClC = Client Count.

Table 6
Graph metrics collected during GVNS for ML-guided selection.

Metric	Description	Abbreviation
Average number of intersections per client	Mean number of route intersections per client, capturing the degree of overlap between different routes.	*Avg Intersections*
Average longest distance between connected clients per route	Mean longest direct Euclidean distance between two consecutive clients within a route, indicating route dispersion.	*Avg Max Distance*
Sum of depot connection distances	Total Euclidean distance between the depot and the first/last clients on all routes.	*Depot Dist*
Average inter-route distance	Mean Euclidean distance between the centers of gravity (centroids) of different routes.	*Avg Route Dist*
Sum of route widths	Total lateral spread of client coordinates within all routes, based on farthest and closest clients from the route centroid.	*Route Width*
Average angular span per route	Mean angular range (in radians) covered by clients within a route, measured in polar coordinates relative to the depot.	*Avg Span Radian*
Compactness by width (normalized distance to gravity line)	Mean distance of clients from the route's gravity line, reflecting spatial cohesion.	*Avg Comp. Width*
Compactness by radian (normalized angular deviation)	Mean angular deviation of clients from the route centroid, reflecting directional cohesion.	*Avg Comp. Radian*
Average depth per route	Mean Euclidean distance of the farthest client from the depot in each route.	*Avg Route Depth*
Standard deviation of clC per route	Measures load imbalance by computing the standard deviation of client allocations across routes.	*Std Dev Clients*
Intersection-based entropy	Measures the uniformity of intersection distribution across routes (structural complexity). Higher entropy indicates even distribution; lower values suggest concentration on a few routes.	*Entropy*

model can adapt its predictions based on evolving trends, thereby enhancing its ability to select the most promising operators over time.

3.2. Data processing and exploratory analysis

We processed the dataset using Scikit-learn [59] for machine learning and R [60] with RStudio [61] for statistical analysis. Preprocessing involved removing missing values and duplicates, dropping low-variance columns (<5%), and scaling features for compatibility with model inputs.

In the exploratory phase, we conducted targeted analyses to understand data characteristics and heuristic operator performance:

- **Correlation Analysis:** Feature relationships were assessed through correlation analysis after preprocessing.
- **Reward Distribution Analysis:** Preliminary analysis (Fig. 2) showed that SH22 and SWAP achieve higher mean improvements with variability, while RL10 and SH21 exhibit lower effectiveness.
- **Temporal Analysis of Heuristic Operator Performance:** Temporal analysis (Fig. 3) revealed:
  - **Early Phase:** High success rates across operators, with SH22 and SWAP being most effective.
  - **Middle Phase:** Success rates decline with increased exploitation; SH22 and CRJN remain effective.
  - **Late Phase:** During convergence, only CRJN and 3OPT maintain relevance.

3.2.1. Exploring feature relationships

The correlation analysis (Fig. 4) reveals interdependencies among graph metrics, UCB-based selections, and historical actions, providing insights for feature selection and model refinement.

Persistence of Success and Failure Trends: Historical action outcomes show strong predictive signals. CRJN exhibits a steady negative correlation with reward, while R2OP shows strong correlation in recent

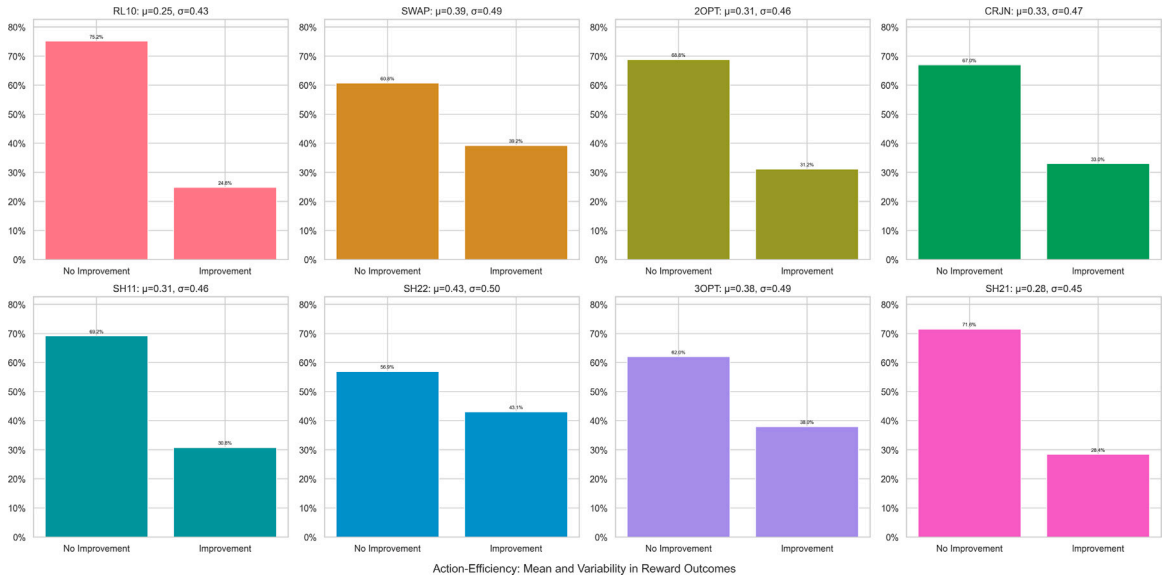


Fig. 2. Reward distribution for each neighborhood operator.

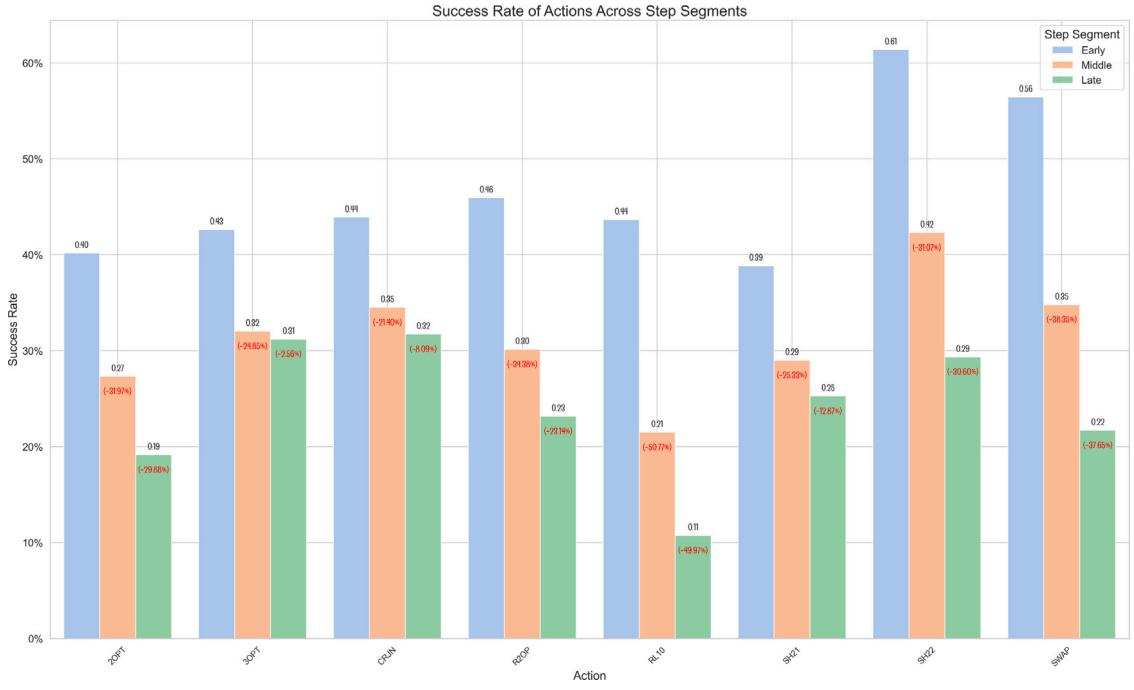


Fig. 3. Success rate of heuristic operators across temporal segments in GVNS.

iterations, which declines over time. In contrast, 2OPT, SWAP, SH22, 3OPT, and SH21 display no consistent relationship with reward.

Temporal Dynamics and Cross-Dependencies: Step correlates positively with past CRJN, RL10, and 2OPT outcomes, highlighting their relevance in later search stages. Entropy shows selective correlations, stronger for recent RL10 actions and more stable for CRJN over time. Older RL10 performance correlates with recent 3OPT and SH21 success rates, suggesting temporal operator dependencies.

UCB Selection and Reward: UCB selections for SWAP, 2OPT, and SH22 strongly correlate with reward, reinforcing their role in action selection. CRJN and SH21 display negative correlations, likely reflecting their use in challenging scenarios rather than intrinsic inefficacy. Despite lower reward alignment, they retain value for identifying suboptimal choices.

Graph Metrics and Reward: `Avg Route Dist` and `Avg Comp Width` show mild correlation with reward, suggesting spatial properties influence outcomes. Step negatively correlates with `Avg Max Distance`, `Avg Route Width`, and `Avg Comp Width`, reflecting solution refinement over time.

These patterns highlight the predictive value of historical actions and UCB-based selections in guiding operator effectiveness. Reward-driven strategies may benefit from actions with stable historical impact while accounting for temporal dependencies. Although graph metrics show mild reward correlation, their influence on search dynamics may extend beyond direct performance, warranting further exploration.

Next, we explore the feature space structure through dimensionality estimation and advanced feature attribution methods.

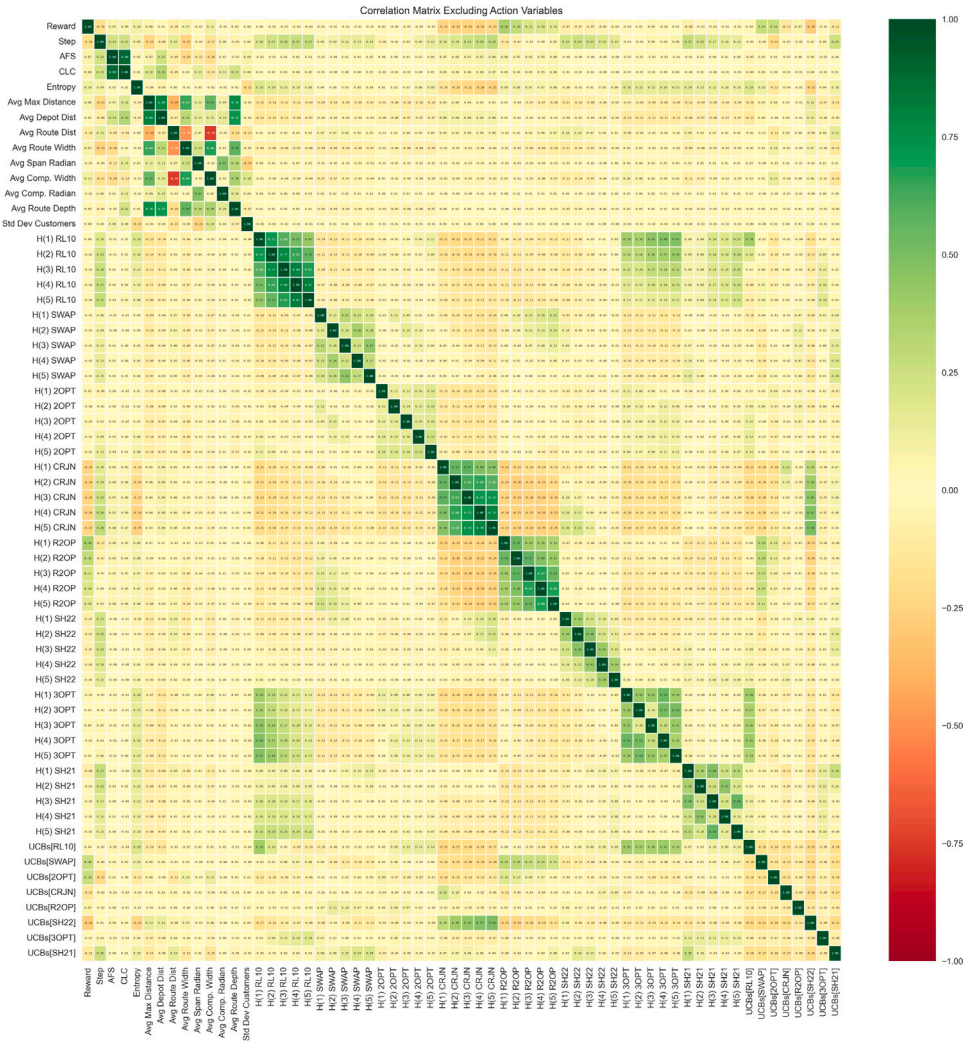


Fig. 4. Correlation matrix showing historical action performance, UCB selections, and search state variable interdependencies.

3.2.2. *Dimensionality estimation and feature relevance analysis*

This analysis helps identify lower-dimensional manifolds and serves as a foundation for refining feature rankings through statistical and ML-based attribution methods.

We applied dimensionality estimation using SCIKIT-DIMENSION [62] to analyze the structure of our feature space and determine how many features are effectively utilized by clustering algorithms. While this estimation primarily aimed to capture structural properties, the autoencoder, as a byproduct, provided feature importance through reconstruction errors.

Table 7 presents the estimated intrinsic dimensions, categorized by method type. Local geometry approaches suggest a low-dimensional manifold (4–8 dimensions), whereas variance-based methods indicate a significantly higher-dimensional structure ($\tilde{5}0$ dimensions). Statistical model selection techniques yield mixed results, while clustering and manifold learning methods suggest a 4D embedding, reflecting a simplified or clustered view.

To further validate these estimates, we applied KNN-based optimization using the Elbow Method [63], Davies–Bouldin Index, and Calinski–Harabasz Index, all of which identified $k = 4$ as the optimal choice (Fig. 6). These analyses provide a basis for subsequent prediction tasks, where machine learning classifiers will be evaluated using PRAUC, with interpretability enhanced through tree explainers and feature importance methods. The identified features and dimensionality estimates serve to highlight the most relevant features, aiding their interpretation within a model-specific context.

We plotted the decision boundaries for SVM and KNN on PCA-reduced data (Fig. 5) to illustrate how different models perceive the data space. SVM shows a smooth boundary, indicating strong generalization, while KNN exhibits an irregular boundary, reflecting sensitivity to local structures. This comparison highlights how model-specific behavior can influence decision-making and classification accuracy when working with complex data structures.

3.2.3. *Feature importance via autoencoder reconstruction error*

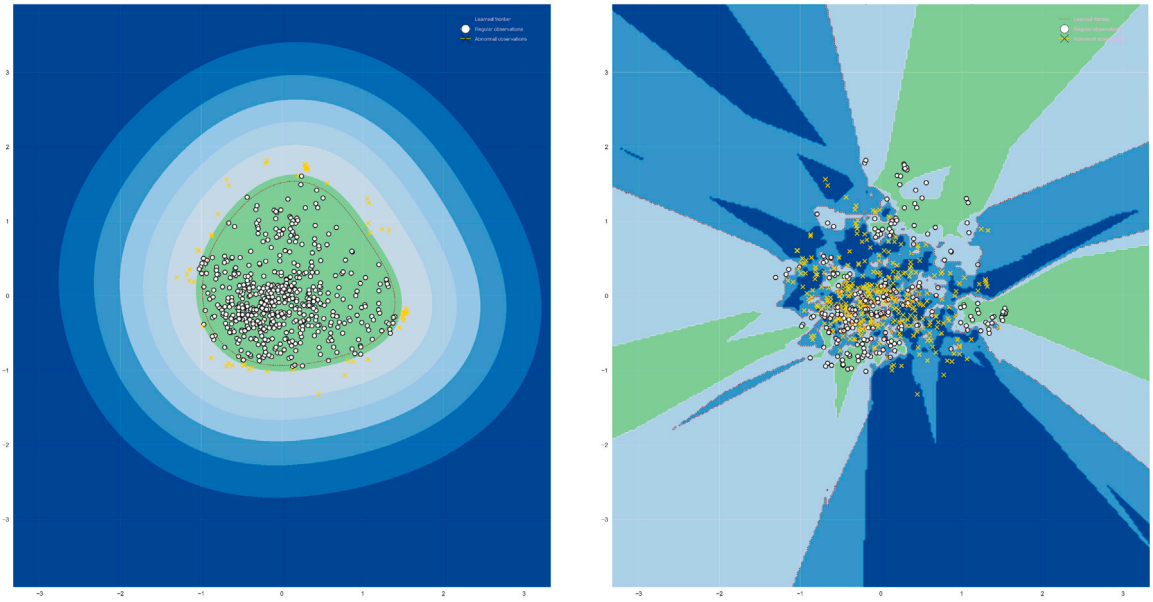
We evaluated feature importance using autoencoder-based reconstruction errors with latent dimensions set to 30, 8, and 4, reflecting variance-based (50D), local geometry (8D), and clustering-based (4D) structures, consistent with Table 7. Higher reconstruction errors reflect greater feature importance due to reconstruction difficulty, indicating the model's reliance on those features for accurate representation [64].

Table 8 shows the top 10 features ranked by reconstruction error across latent dimensions. *Outcome History* and *Search State* features increase in importance at lower dimensions, suggesting that temporal and state variables are more predictive when the model is forced to rely on compressed information. Conversely, *Graph Metrics* retain greater significance at higher dimensions, likely because structural properties require more complex representations to capture spatial relationships. Features, such as *Step* and *AFS* remain consistently important.

Features like *Step* and *AFS* remain critical across dimensions, while outcome history and search-state variables increase in importance in

Table 7
Estimated intrinsic dimensions using various methods.

Method	Estimated dimension	Method type
Principal component analysis (95%)	53.00	Variance-based
Bayesian optimization	50.00	Variance-based
Gap statistic	48.00	Variance-based
Expectation maximization	43.00	Model selection
Akaike information criterion	39.00	Model selection
Bayesian information criterion	7.00	Model selection
Dimension adaptive neighborhood component	8.02	Local geometry
Max likelihood estimation for intrinsic dimension	7.70	Local geometry
Max likelihood estimation	7.49	Local geometry
Correlation integral method	4.75	Local geometry
Isometric mapping	4.00	Manifold learning
Silhouette score analysis	4.00	Clustering-based



(a) SVM Decision Boundary in PCA Space (b) KNN Decision Boundary in PCA Space

Fig. 5. Decision Boundaries of SVM and KNN in PCA Space. Regular observations are marked with circles, while outliers are marked with yellow crosses. The decision boundary (dashed line) separates normal observations from outliers, illustrating the classification boundaries as determined by the respective models.

compact spaces. Graph metrics are more valuable in higher dimensions, reflecting their structural relevance.

3.3. Feature contribution analysis

We applied four statistical methods, Chi-Squared, Pearson, Mutual Information, and LASSO, to assess feature importance and improve feature selection (see Table 9). Chi-Squared measures categorical dependency, Pearson captures linear relationships, Mutual Information reflects both linear and non-linear dependencies, and LASSO reduces collinearity for a more compact feature set. Features were grouped into Outcome History, UCB Selection, Search State, Graph Metrics, and Actions.

Outcome History ranked highest across methods, confirming its strong predictive value. UCB Selection ranked second across all methods (19.65–27.72), highlighting its consistent predictive strength. Search State ranked low under Chi-Squared, Pearson and LASSO (0.51–5.22) but gained relevance under Mutual Information (11.45), reflecting its impact in dynamic search scenarios. Graph Metrics were important in high-dimensional settings, with a total contribution of 56.72 under autoencoder-based analysis, but only ranked fourth or fifth across all the four statistical methods (Table 10).

Top predictors like *H(1) CRJN* consistently appeared in the top five across methods, reinforcing the stability of Outcome History and UCB Selection (Fig. 7). While individual Graph Metrics ranked lower in the raw analysis, their combined contribution was significant, ranking third under Pearson and fourth or fifth in the other methods. Univariate methods highlighted individual predictor strength, while LASSO produced a more compact set, showing the benefit of combining detailed and aggregated analysis.

3.4. Overview of machine learning techniques used

We employed twelve supervised machine learning models to predict promising neighborhoods within the VNS framework, trained on data derived from GVNS runs on CVRP instances.

3.4.1. Machine learning model classification and selection

The selected models span six categories: instance-based, linear, non-linear, tree-based, boosting, and artificial neural networks (Table 11). This diverse set was chosen to address varying data complexities and achieve robust predictive performance, but also to gain deeper insight into the features and patterns that drive each algorithm's predictions. In particular, we aimed to discern the specific attributes that each model considers most informative, thereby enriching our understanding of the data's underlying structure.

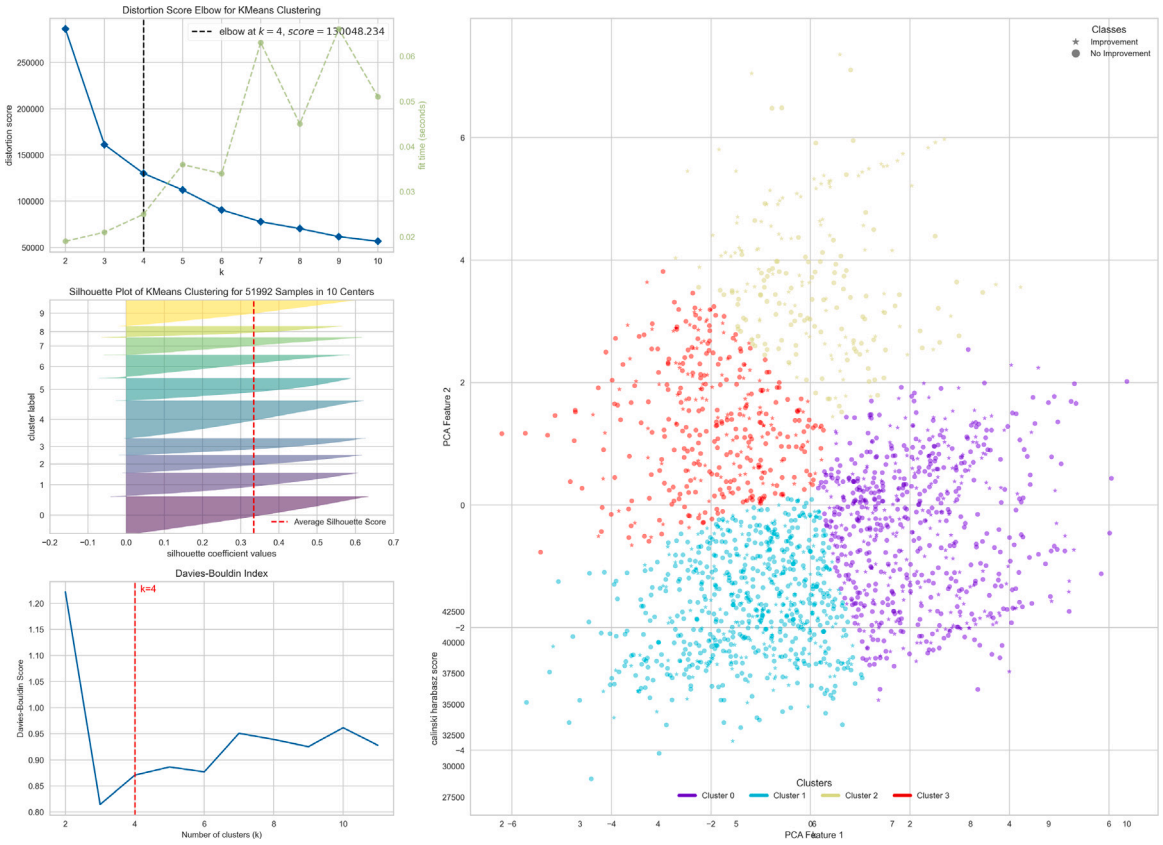


Fig. 6. Optimal Cluster Determination using Elbow Method, Silhouette Analysis, Davies–Bouldin Index, and Calinski–Harabasz Index.

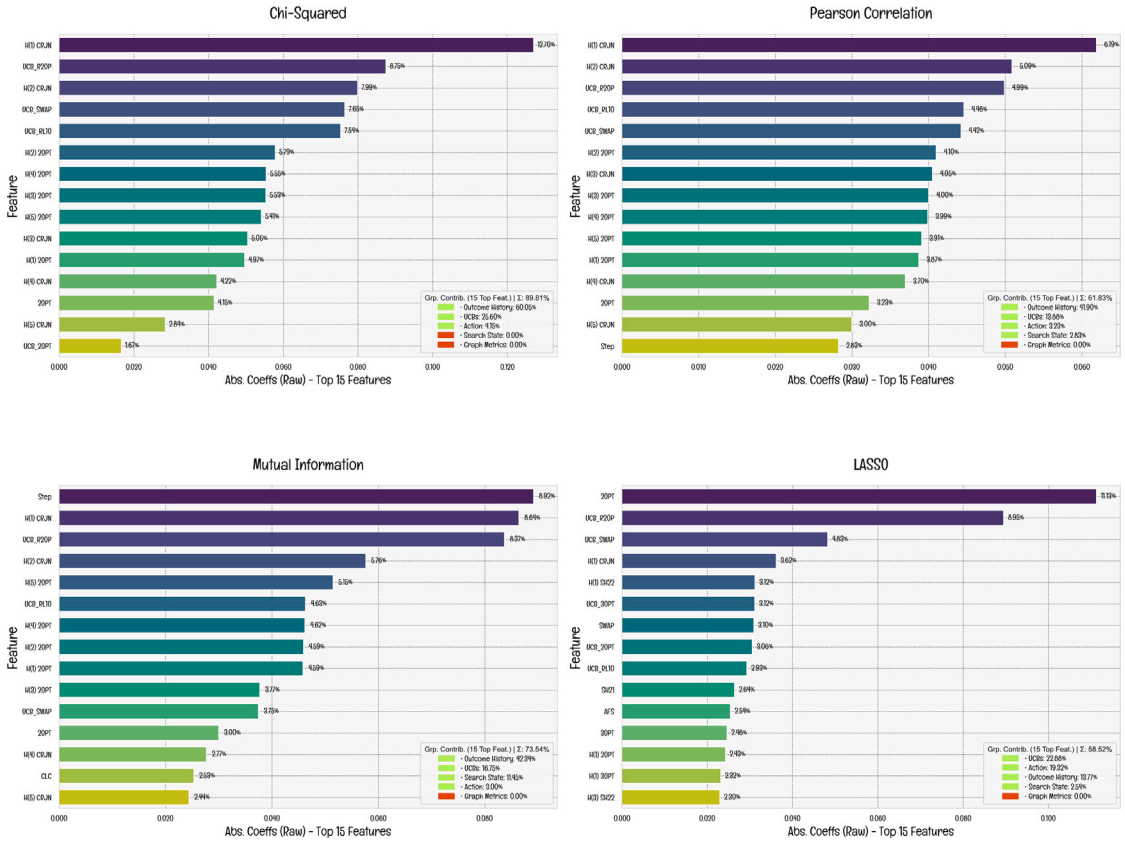


Fig. 7. Raw feature importance across analytical methods.

Table 8
Top 10 features by autoencoder reconstruction error across latent dimensions.

ID	Feature	Type	Reconstruction error (%)
1	Std Dev Clients	Graph Metrics	11.82
2	Avg Comp. Radian	Graph Metrics	9.26
3	Avg Span Radian	Graph Metrics	7.98
4	Step	Search State	7.69
5	Avg Route Width	Graph Metrics	7.46
6	Avg Route Dist	Graph Metrics	7.38
7	AFS	Search State	6.76
8	Avg Depot Dist	Graph Metrics	6.76
9	CLC	Search State	6.12
10	Avg Comp. Width	Graph Metrics	6.06
Sum of explained variance from top 10 features (latent = 30)			***77.29***
ID	Feature	Type	Reconstruction error (%)
1	CLC	Search State	7.31
2	Entropy	Graph Metrics	7.05
3	AFS	Search State	4.61
4	Avg Route Depth	Graph Metrics	4.38
5	Avg Comp. Width	Graph Metrics	3.90
6	Std Dev Clients	Graph Metrics	3.38
7	Avg Depot Dist	Graph Metrics	3.35
8	Avg Route Dist	Graph Metrics	3.34
9	H(4) CRJN	Outcome History	3.23
10	H(2) CRJN	Outcome History	3.16
Sum of explained variance from top 10 features (latent = 8)			***43.71***
ID	Feature	Type	Reconstruction error (%)
1	Entropy	Graph Metrics	5.93
2	CLC	Search State	4.97
3	H(2) CRJN	Outcome History	3.41
4	H(3) CRJN	Outcome History	3.31
5	H(1) CRJN	Outcome History	3.17
6	AFS	Search State	3.08
7	Avg Route Depth	Graph Metrics	2.77
8	R2OP	Outcome History	2.66
9	H(5) CRJN	Outcome History	2.57
10	Avg Comp. Width	Graph Metrics	2.49
Sum of explained variance from top 10 features (latent = 4)			***34.36***

Table 9
Aggregated feature importance rankings for top 15 features.

Chi-Squared		Pearson		Mutual info		LASSO	
Outcome History	65.65	Outcome History	58.97	Outcome History	52.53	Outcome History	42.64
UCB Selection	27.72	UCB Selection	19.65	UCB Selection	20.55	UCB Selection	23.61
Action	5.49	Graph Metrics	8.10	Search State	11.45	Action	22.33
Graph Metrics	0.64	Action	8.06	Graph Metrics	9.35	Graph Metrics	6.76
Search State	0.51	Search State	5.22	Action	6.11	Search State	4.66

Table 10
Comparison of feature importance across methods.

Feature group	Autoencoder	Chi-Squared	Pearson	Mutual info	LASSO
Outcome History	**Moderate** but widespread	**High** (65.65)	**High** (58.97)	**High** (52.53)	**High** (42.64)
UCB Selection	–	**Moderate** (27.72)	**Moderate** (19.65)	**Moderate** (20.55)	**Moderate** (23.61)
Search State	**Moderate** at low dimensions	Low (0.51)	Low (5.22)	**Moderate** (11.45)	Low (4.66)
Graph Metrics	**Moderate** at high dimensions	Low (0.64)	**Moderate** (8.10)	**Moderate** (9.35)	**Moderate** (6.76)

Tree-based models (Decision Tree [65,66], Random Forest [67–69], Extra Trees [70]) require minimal feature engineering. Boosting methods (Gradient Boosting, XGBoost [71], LightGBM [72], CatBoost [73], AdaBoost [74,75]) iteratively refine weak learners. Complementary baseline models include Support Vector Machines (SVM) with an RBF kernel, K-Nearest Neighbors (KNN), and Logistic Regression (LGR), while Multilayer Perceptrons (MLP) capture complex non-linear relationships.

3.4.2. Model training and performance metrics

Each classifier was trained to predict optimal neighborhood operators. We evaluated performance using Accuracy, Precision, Recall,

Table 11
Categorization of supervised machine learning algorithms by complexity.

Category	Methods	Speed	Interpretability	Train complexity	Predict complexity
Instance-based	KNN	Very fast	Low	$O(1)$	$O(nd)^*$
Linear	LGR	Fast	Low	$O(I\,n\,d)$	$O(d)$
Non-linear	SVM (RBF)	Very slow	Modest	$O(n^2\,d)$	$O(n\,d)$
Tree-based	Decision Tree	Fast	High	$O(n\,d\,\log n)$	$O(\log n)$
	Random Forest	Modest	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
	Extra Trees	Fast	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
Boosting	GBM	Slow	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
	XGBoost	Modest	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
	LightGBM	Fast	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
	CatBoost	Modest	Modest	$O(m\,n\,d\,\log n)$	$O(m\,\log n)$
	AdaBoost	Slow	Modest	$O(m\,n\,d)$	$O(m)$
ANN	MLP	Slow	Low	$O(I\,n\,d\,h\,o)$	$O(d\,h\,o)$

Parameters: n = samples, m = trees, d = features, h = hidden layers, o = outputs, I = iterations.
For KNN, complexity is $O(nd)$ (brute force) or $O(\log n)$ using KD-/Ball-Tree [76].
References: Tree-based models [67,70,71]; MLP [77]; boosting vs. bagging [78].

F1-Score, *Area Under the Receiver Operating Characteristic* (AUC-ROC), and *Precision–Recall Area Under Curve* (PR-AUC). Given the imbalanced nature of the dataset, where successful operator selections are sparse, we prioritized PR-AUC as the primary metric, as it better captures the model's ability to identify minority-class instances. While the overall dataset is not extremely sparse in terms of the binary target variable, temporal sparsity emerges as the search progresses. Initially, 33% of instances correspond to successful moves, but this proportion declines as improvements become rarer. This poses a challenge for learning reliable decision patterns, requiring the model to generalize from earlier stages while adapting to the increasingly constrained search space.

3.5. Hyperparameter optimization and standardization

We tuned hyperparameters using Bayesian optimization via Optuna [79], optimizing PR-AUC to address dataset imbalance. A stratified 10-fold cross-validation balanced efficiency and robustness, while each model was tuned according to its architecture to reduce bias from default settings. See Appendix. Hyperparameters for details about the search space and optimal configurations.

The tuning process significantly enhanced model stability and predictive accuracy by minimizing biases from default settings and establishing a standardized performance baseline. The optimized hyperparameters allowed for fair comparison across classifiers, ensuring that each model operated under its best configuration before benchmarking in the ML-guided GVNS framework.

3.5.1. Comparative performance of ML classifiers

We partitioned the dataset, collected during the Data Collection Phase (Fig. 1) from the A-Set runs, into 80% training and 20% holdout subsets, ensuring no duplicate rows in either split. This provides sufficient training data while preserving an unseen subset for unbiased evaluation. Table 12 summarizes the performance of the classifier in the holdout subset, ranked by PR-AUC.

LightGBM achieves the highest PR-AUC (0.9398) and ranks best in Accuracy, Recall, F1, and AUC-ROC. Gradient Boosting records the highest Precision (0.8812), but its lower Recall reduces F1-score and PR-AUC. Other classifiers show progressively lower PR-AUC, highlighting the need to balance precision and recall in imbalanced settings. Random Forest ranks second in PR-AUC (0.8281), followed by Extra Trees (0.8139), CatBoost (0.8099), MLP (0.8085), and Gradient Boosting (0.8044).

In summary, LightGBM's strength in detecting minority-class instances makes it well-suited for guiding the VNS framework. These results suggest that while some models excel in individual metrics, PR-AUC remains the most reliable assessment for imbalanced tasks.

3.6. Feature importance evaluation across machine learning models

We evaluated feature contributions using four interpretability methods: Mean Decrease Impurity (MDI), Permutation Importance, SHAP (SHapley Additive exPlanations), and TreeInterpreter. Each method provides unique insights:

- **MDI**: Measures impurity reduction at tree splits; efficient but biased toward high-cardinality features [80].
- **Permutation Importance**: Quantifies performance degradation when feature values are shuffled; robust to correlations but computationally intensive [67].
- **SHAP**: Uses Shapley values to explain global and local feature contributions; computationally expensive but highly interpretable [81].
- **TreeInterpreter**: Provides intuitive, tree-specific explanations; less comprehensive than SHAP but computationally efficient [82].

We analyzed the top three classifiers, LightGBM, Random Forest, and Extra Trees, based on their PR-AUC rankings (Table 12). Tables 13–15 summarize feature importance results, consistently highlighting Outcome History, Graph Metrics, and UCBs as the most influential features. These findings underscore the critical roles of temporal dependencies, spatial structures, and exploration–exploitation mechanisms in guiding GVNS optimization.

While TRACE was implemented within GVNS in this study, the TRACE architecture is agnostic to the metaheuristic framework. Appendix 5 and Appendix 6 provide pseudocode illustrations of how TRACE can be mapped onto Simulated Annealing and Genetic Algorithms, using the same operator selection mechanism with minimal architectural changes. For Simulated Annealing, Table C.20 summarizes the integration points, model usage cycle, and key contextual features extracted per move.

4. Results

This section presents the experimental setup, comparative performance analysis, and discussion of insights gained from the ML-guided GVNS framework.

4.1. Computational setup

All experiments were conducted on an Intel Core Ultra 9 285 CPU, 128 GB RAM, and an NVIDIA RTX 3090 GPU running Windows 11. Python 3.11 was used with scikit-learn and Pandas.

We used a cooperative parallel GVNS model based on [14] to improve computational efficiency. One thread applies ML-guided operator selection while others run conventional GVNS iterations, enabling solution sharing across threads. This multi-threaded structure accelerates

Table 12
Classifier performance summary for Accuracy, Precision, Recall, F1-score, AUC-ROC, and PR-AUC. Gray-shaded cells indicate the best performance per metric.

Classifier	Accuracy	Precision	Recall	F1	AUC-ROC	PR-AUC
LightGBM	0.8813	0.7485	0.9600	0.8411	0.9702	0.9398
Random Forest	0.8304	0.8192	0.6185	0.7048	0.8832	0.8281
Extra Trees	0.8315	0.7959	0.6527	0.7172	0.8788	0.8139
CatBoost	0.8313	0.7962	0.6514	0.7166	0.8774	0.8099
MLP	0.8210	0.7378	0.7029	0.7199	0.8830	0.8085
Gradient Boosting	0.8187	0.8812	0.5156	0.6506	0.8751	0.8044
XGBoost	0.8175	0.7212	0.7214	0.7213	0.8830	0.8017
Decision Tree	0.8134	0.7050	0.7393	0.7218	0.8616	0.7596
Naive Bayes	0.7243	0.5656	0.6803	0.6177	0.7642	0.6830
Logistic Regression	0.7452	0.5955	0.6911	0.6398	0.8061	0.6658
AdaBoost	0.6904	0.5821	0.1931	0.2900	0.7396	0.5864
KNN	0.7067	0.5659	0.4477	0.4999	0.6971	0.5853

All classifiers were trained on the same dataset split.

Table 13
Top LightGBM features by importance (normalized contributions %).

LightGBM					
MDI	Value (%)	Permutation	Value (%)	SHAP	Value (%)
Graph Metrics	59.04	Graph Metrics	23.56	Outcome History	34.56
Outcome History	18.16	Outcome History	21.91	Search State	19.28
Search State	11.07	Search State	15.47	Graph Metrics	18.40
UCBs	7.63	UCBs	21.62	UCBs	18.15
Action	4.10	Action	17.44	Action	9.60

Table 14
Top Random Forest features by importance (normalized contributions %).

Random Forest					
MDI	Value (%)	SHAP	Value (%)	TreeInterpreter	Value (%)
Graph Metrics	52.94	Outcome History	40.49	Graph Metrics	39.16
Outcome History	27.38	Graph Metrics	38.50	Outcome History	39.00
UCBs	6.54	UCBs	7.23	UCBs	7.34
Action	5.48	Action	6.67	Action	6.76
Search State	7.66	Search State	7.10	Search State	7.74

Table 15
Top Extra Trees features by importance (normalized contributions %).

Extra Trees					
MDI	Value (%)	SHAP	Value (%)	TreeInterpreter	Value (%)
Outcome History	44.06	Outcome History	54.88	Outcome History	51.08
Graph Metrics	28.55	UCBs	16.57	Graph Metrics	20.47
UCBs	12.21	Graph Metrics	13.39	UCBs	13.63
Action	9.00	Action	11.06	Action	9.59
Search State	6.17	Search State	4.10	Search State	5.23

convergence and enhances solution quality by combining data-driven and traditional search strategies, facilitating direct comparison between ML-guided and conventional GVNS performance. All algorithms were run for 30 iterations.

4.2. TRACE-VNS integration framework

The ML-enhanced GVNS framework dynamically selects neighborhood operators based on learned predictions, improving adaptability and solution efficiency. TRACE leverages predictive search guidance to select neighborhood operators based on contextual features extracted from the GVNS state. The integration process follows four key steps, as illustrated in Fig. 8: The ML inference component adds negligible runtime overhead. Offline model integration ensures deterministic inference with negligible runtime latency, aligning with industrial deployment constraints. Measured across multiple instances, the average per-iteration prediction latency was $3.2\,\text{ms} \pm 0.6\,\text{ms}$, while the GVNS step duration averaged $11.5\,\text{ms} \pm 1.1\,\text{ms}$. These values confirm that the model-guided process is real-time compatible and preserves optimization speed.

1. **Feature Vector Construction:** At each GVNS iteration, a feature vector is built from graph metrics, operator history, UCB values, and problem-specific variables.
2. **Prediction and Operator Selection:** The ML model predicts operator success probability, selecting the most promising one.
3. **Parallel Execution:** ML guidance is applied to one thread, while others run standard GVNS, reducing overhead and allowing solution sharing.
4. **Solution Sharing:** High-quality solutions from the ML-guided thread are shared with others, balancing exploration and exploitation.

4.3. Comparison with conventional GVNS

In this section, we evaluate the generalization of TRACE-VNS using the top three models (LightGBM, Random Forest, Extra Trees) and compare their impact on solution quality and computational time.

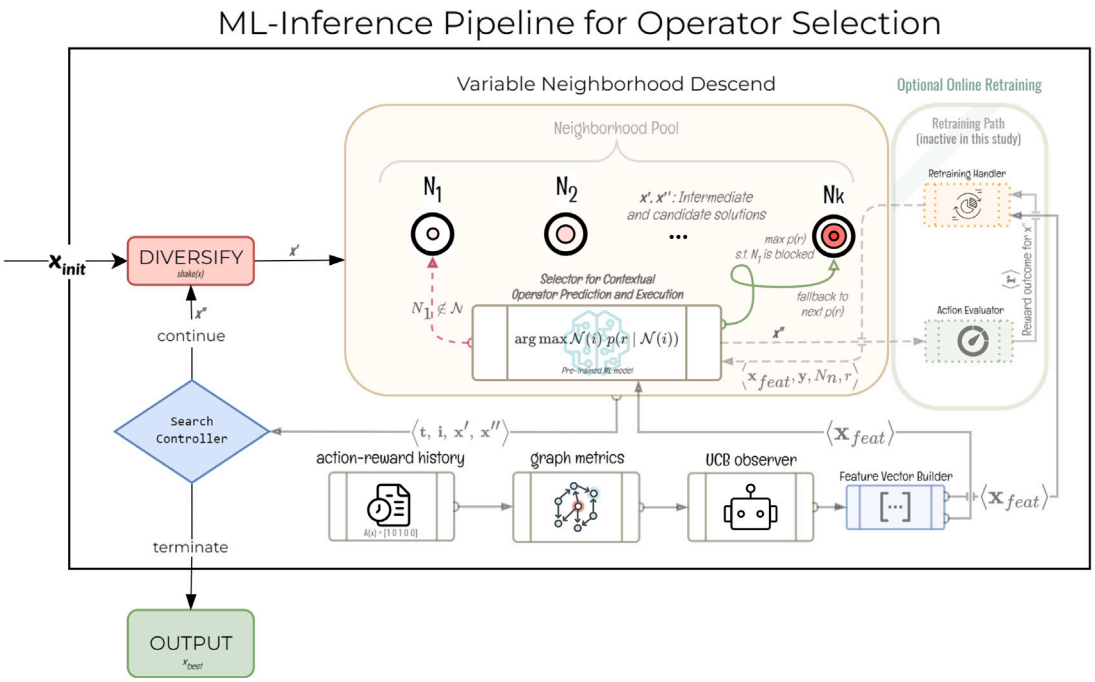


Fig. 8. ML-Inference Pipeline for Operator Selection in TRACE-VNS. The GVNS current search state is encoded into a feature vector x_{feat} composed of action-reward history, graph metrics, and UCB indicators. This vector is passed to a pre-trained ML model, which ranks neighborhood operators based on predicted improvement likelihood. If the top-ranked operator is blocked, the system automatically falls back to the next-best ranked candidate, ensuring seamless continuation of the search process. The selected operator is executed, and if retraining were enabled, the resulting tuple $\langle x_{\text{feat}}, y, N_n, r \rangle$ could be stored for online adaptation. In this study, retraining is inactive.

4.3.1. *Comparison of solution quality and generalization performance*

To assess generalization, the top three ML models (trained and evaluated on A-Set) were tested on B-Set and X-Set CVRP instances, which differ in depot location, customer distribution, and problem scale. TRACE-VNS maintained predictive accuracy across these diverse settings, demonstrating robustness and adaptability through successful transfer learning. As performance differences per instance were approximately symmetric and near-normal (Shapiro–Wilk test), we applied the Wilcoxon signed-rank test, a standard non-parametric test for paired, matched-instance data [83,84]. Reported p-values in Table 16 indicate significance between the best ML model and conventional GVNS.

Table 16 summarizes average solution quality (error %) and relative improvement across all instances. TRACE-VNS achieved cost reductions of 0.5–5.5%, underperforming in only one case. The *relative improvement (%)* column also reflects convergence speed (after 30 iterations), ranging from −10.14% to 86.07%, with an average gain of 31.11%.

Fig. 9 reveals that larger instances tend to yield higher errors across all methods, with Extra Trees and GVNS showing the strongest size-error correlation. The right panel further confirms that all models exhibit increased error on the more complex X-Set, underscoring the generalization challenge across structurally distinct instances.

Our approach builds on the same modular framework as the RL-based Bandit VNS [8], with both leveraging adaptive operator selection. While Bandit VNS results are included in Table 16 for context, differences in objectives, operator sets, and runtime environments make direct comparison difficult. Still, both consistently outperform classical GVNS, underscoring the value of learning-driven guidance.

- **Greater gains in small/medium instances**: ML-guided methods outperform GVNS more noticeably in smaller instances, likely due to training on A-Set (up to 80 clients). In larger instances, model effectiveness decreases as graph structures and historical patterns become less representative.
- **Smaller but consistent gains in larger instances**: ML-guided methods yield stable improvements, countering overfitting concerns.
- **Model specialization**: Extra Trees and Random Forest occasionally outperform LightGBM, suggesting adaptation to specific problem features.
- **No consistent dominance**: No single model or GVNS dominates across all instances; best performer varies by the characteristics of the instance.
- **GVNS remains competitive**: GVNS outperforms ML models in some smaller instances, confirming the strength of its neighborhood operators in certain structures.

4.4. *Retrainability: Update modes and deployment tradeoffs*

The TRACE framework uses offline-trained models for deterministic, low-latency inference. However, retrainability becomes essential as data distributions shift. Retraining can occur offline (e.g., batch updates) or online (e.g., triggered by drift or degraded performance [8]). The system supports contextual feedback collection and model updates without interfering with core inference.

Retrainability includes: (i) *incremental updates*, which adjust model parameters using new data batches (e.g., `partial_fit` in linear models and neural nets); (ii) *warm-start*, which resumes training from a saved state (e.g., `init_model`); and (iii) *full retraining*, which rebuilds the model from scratch when no update interface is available.

These options differ in infrastructure needs, latency tolerance, and resilience to drift. Table 17 summarizes retraining capabilities and update APIs. Offline retraining provides a practical middle ground between static policies and fully online learning.

5. Discussion

Our experiments show promising performance when combining structural, temporal, and exploration-driven features when selecting neighborhood operators. Outcome history captured much of the predictive signal in SHAP and TreeInterpreter analyses, indicating strong direct effects on individual predictions, especially in Extra Trees where it ranked highest.

Table 16
Instance-level solution quality for EXTR, LGBM, RFOR, and VNS, reported as average error (%) from the optimum after 30 iterations. Best and worst results are in green and red. *Relative Improv. (%)* $= (E_{\text{VNS}} - E_{\text{Best}})/E_{\text{VNS}} \times 100$, reflecting both error reduction and convergence speed. RL results [8] are shown for reference but not directly comparable due to differing objectives, GVNS setups, and environments.

		Solution quality (avg. error gap %)				Offline ML vs. conventional VNS vs. Online RL		
Instance	Optimal	EXTR	LGBM	RFOR	VNS	Offline-ML Rel.Imp.(%)	p-value	Online-RL Rel.Imp.(%)
B-n31-k5	672	1.06	**1.05**	1.24	1.17	10.26	0.0056	35.09
B-n34-k5	788	1.72	1.27	**1.13**	2.02	44.06	0.0053	12.03
B-n35-k5	955	1.71	**1.60**	2.61	2.06	22.33	0.0029	12.91
B-n38-k6	805	0.96	0.76	0.84	**0.69**	−10.14	0.0075	−33.30
B-n39-k5	549	3.40	1.97	**1.77**	3.50	43.71	0.0015	−17.77
B-n41-k6	829	1.91	3.23	**1.73**	2.49	30.52	0.0076	6.26
B-n43-k6	742	**1.09**	1.35	1.34	1.84	40.76	0.0050	0.74
B-n44-k7	909	2.80	**2.11**	2.55	2.58	18.22	0.0077	19.92
B-n45-k5	751	1.85	**1.65**	2.27	3.64	54.67	0.0038	22.94
B-n45-k6	678	**4.95**	5.35	5.28	5.92	10.81	0.0049	0.97
B-n50-k7	741	1.14	**0.37**	0.54	1.25	70.40	0.0019	49.87
B-n50-k8	1312	1.27	1.43	**1.20**	1.38	13.04	0.0070	9.63
B-n51-k7	1032	1.60	**0.63**	0.67	4.52	86.07	0.0005	N/A
B-n52-k7	747	**1.51**	2.12	1.66	4.07	62.90	0.0070	10.16
B-n56-k7	707	2.84	**1.16**	1.21	2.56	54.69	0.0003	18.93
B-n57-k7	1598	2.78	**1.03**	2.45	6.53	84.23	0.0078	N/A
B-n57-k9	1598	**2.42**	2.77	2.89	2.85	15.09	0.0022	22.28
B-n63-k10	1496	4.86	**4.73**	5.15	5.23	9.56	0.0058	8.14
B-n64-k9	861	4.54	4.78	**4.06**	4.40	7.73	0.0034	16.13
B-n66-k9	1316	**2.48**	3.37	2.70	3.14	21.02	0.0071	14.88
B-n67-k10	1032	**4.25**	4.76	4.43	5.13	17.17	0.0075	9.28
B-n68-k9	1272	3.66	**3.03**	3.69	3.85	21.32	0.0096	5.16
B-n78-k10	1221	**4.97**	5.00	5.19	6.81	27.43	0.0053	2.38
X-n101-k25	27 591	**3.76**	3.81	3.93	4.31	12.77	0.0064	15.04
X-n106-k14	26 362	2.65	**2.56**	2.70	3.07	16.62	0.0847	8.06
X-n110-k13	14 971	4.26	4.44	**4.09**	4.38	6.62	0.0764	5.05
X-n120-k6	13 332	5.78	5.71	**4.55**	8.19	44.44	0.0480	8.91
X-n148-k46	43 448	3.94	**3.31**	3.25	3.61	9.97	0.0156	18.09
X-n256-k16	18 839	6.20	**4.75**	5.36	8.66	45.14	0.0410	N/A
Average performance (%)		2.97	2.76	2.77	3.78	31.11		10.69

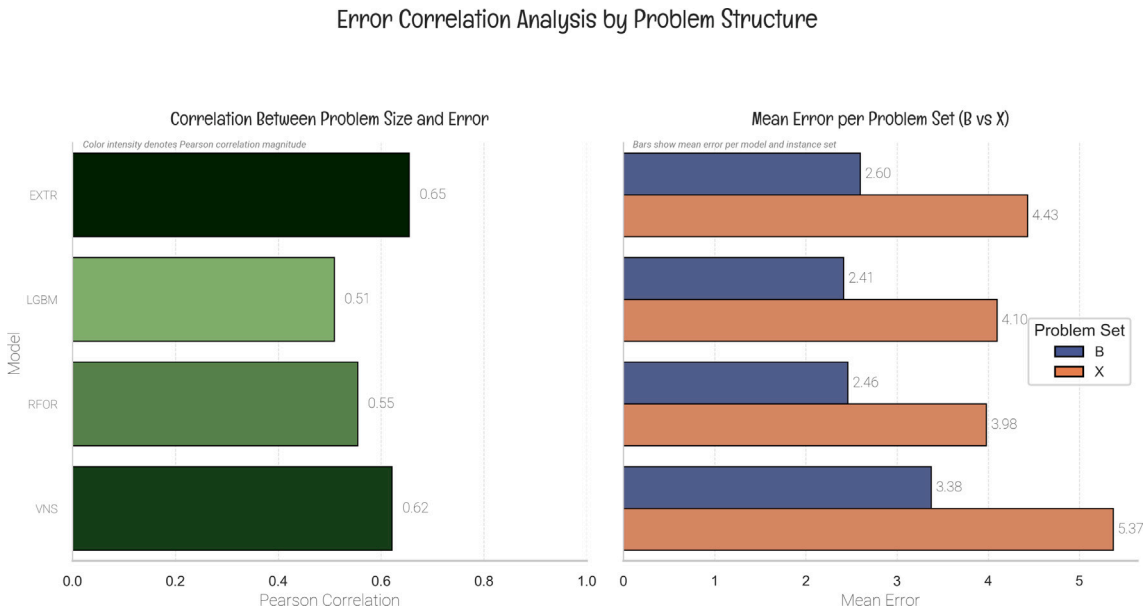


Fig. 9. Error correlation analysis by problem structure. **Left** - Correlation between problem size and error: Pearson correlation between the number of nodes and error per model. Bar color intensity reflects correlation strength (darker = higher). **Right** - Mean error per problem set (B vs. X): Average error for each model on structurally distinct instance groups. Blue denotes B-Set and orange denotes X-Set.

Table 17
Retraining support: full, partial, or none. Models without support require full retraining.

Algorithm	Type	Incremental update	Warm-start support	Notes
Instance-based				
KNN	Instance-based	✗	✗	Full retrain required
Linear				
Logistic Regression	Linear	✓	✗	Supports partial fit (online)
Non-linear				
SVM (RBF)	Non-linear	✗	✗	Full retrain required
Tree-based				
Decision Tree	Tree-based	✗	✗	Full retrain required
Random Forest	Tree-based	✗	✗	Full retrain required
Extra Trees	Tree-based	✗	✗	Full retrain required
Boosting				
GradientBoosting	Boosting	✗	△	Estimator extension; lacks full continuation
XGBoost	Boosting	✗	✓	xgb model resumes training (offline)
LightGBM	Boosting	✗	✓	init model resumes training (offline)
CatBoost	Boosting	✗	✓	Resume via init model (offline)
AdaBoost	Boosting	✗	✗	Full retrain required
Neural Net				
MLP (ANN)	Neural Net	✓	✗	Supports partial fit (online)

Models with partial fit support incremental updates. Warm-start resumes via init model; otherwise, full retraining is needed. (Online): from streams, (Offline): from checkpoints.

Table B.18
Summary of key hyperparameter search spaces and optimal configurations.

Algorithm	Key hyperparameters (Search space)	Optimal
Logistic Regression	C : [0.05, 0.9] (step: 0.1); MaxIter: [1000, 5000] (step: 500); Solver: {newton-cg, lbfgs, liblinear, sag, saga}	$C = 0.09$, MaxIter = 1500; Solver = sag, Penalty = l_2
Decision Tree	MaxDepth: [3, 20, None]; MinSplit: {2, 5, 10}; MinLeaf: {1, 2, 5, 10}	MaxDepth = 10, MinSplit = 5; MinLeaf = 2, Criterion = gini
Random Forest	Estimators: [100, 1000] (step: 200); MaxDepth: [3, 20, None] (step: 1); Max Features: {sqrt, $\log_2$}	Estimators = 100, MaxDepth = None; MinSplit = 10, Max Features = sqrt
Extra Trees	Estimators: [500, 1100] (step: 250); MaxDepth: [3, 30] (step: 5); MinSplit: {2, 5, 10}	Estimators = 500, MaxDepth = 20; MinSplit = 2, Max Features = sqrt
AdaBoost	Estimators: [50, 500] (step: 50); LR: [0.01, 1] (step: 0.01); Base Depth: [2, 15] (step: 1)	Estimators = 100, LR = 1; Base = Tree(max_depth = 5)
Gradient Boosting	Estimators: [150, 2000] (step: 100); LR: {0.01, 0.03, 0.1}; MaxDepth: [3, 20]	Estimators = 800, LR = 0.03; MaxDepth = 5, MinSplit = 2
XGBoost	Estimators: [150, 500] (step: 50); LR: [0.01, 0.5] (step: 0.1); MaxDepth: [2, 12] (step: 2)	Estimators = 250, LR = 0.01; MaxDepth = 8
LightGBM	LR: {0.01, 0.03, 0.1}; Num Leaves: [5, 100] (step: 10); MaxDepth: {−1, 5, 10, 15}	LR = 0.03, Leaves = 31; MaxDepth = 10
CatBoost	MaxLeaves: [31, 255] (step: 2); Iterations: [100, 10 000] (step: 100); Depth: [5, 20] (step: 5)	MaxLeaves = 63, Iterations = 2000; Depth = 10
KNN	Neighbors: [3, 30] (step: 2); LeafSize: [10, 100] (step: 10); p: {1, 2}	Neighbors = 8, LeafSize = 50; $p = 2$
SVM	C : [0.1, 100] (step: 10); Gamma: {auto, scale}; Kernel: {linear, rbf}	$C = 1.5$, Gamma = scale; Kernel = rbf

Graph metrics, despite low standalone correlation with the target, consistently ranked among the top features across all attribution methods. MDI placed them first in LightGBM (59%) and Random Forest (53%) (Tables 13 and 14), suggesting these models use graph features for early splits due to their impurity-reducing capacity. Extra Trees (Table 15) instead prioritized outcome history. This reveals a model-specific trade-off: some algorithms favor structural signals, others emphasize behavioral trends. SHAP, TreeInterpreter, and Permutation analyses confirmed the contextual value of graph features. Their contribution came primarily through interaction effects, not direct

Table B.19
Summary of feature notations used in TRACE-VNS.

Notation	Description
H(1)–H(5) RL10	Last 5 outcomes for RL10 operator
H(1)–H(5) SWAP	Last 5 outcomes for SWAP operator
H(1)–H(5) 2OPT	Last 5 outcomes for 2OPT operator
H(1)–H(5) CRJN	Last 5 outcomes for CRJN operator
H(1)–H(5) R2OP	Last 5 outcomes for R2OP operator
H(1)–H(5) SH22	Last 5 outcomes for SH22 operator
H(1)–H(5) 3OPT	Last 5 outcomes for 3OPT operator
H(1)–H(5) SH21	Last 5 outcomes for SH21 operator
UCBs[RL10–SH21]	UCB selection indicators per operator

correlation. Removing them based solely on correlation would discard features supporting motif patterns, subgraph synergies, and structural context, consistent with prior findings [85–89].

UCB-based indicators ranked low in MDI but much higher in permutation importance, implying they shape downstream behavior without dominating splits. Action features remained mid to low across all methods, helping refine decisions without driving them. Search state showed model- and depth-dependent importance, contributing in selective conditions. Altogether, models interpret features differently based on algorithm and attribution method. Structure, behavior, and exploration each play distinct but complementary roles in operator selection.

Dimensionality analysis revealed a latent space of 4 to 8 dimensions (Table 7), confirming that most predictive variation lies on a small subspace despite the 69 input features. This explains why compressed representations preserved sensitivity to features like Search State and why models consistently focused on a narrow subset. The divergence between variance-based and manifold-based estimates clarifies model biases: MDI favored high-variance graph metrics, while SHAP and permutation methods emphasized behavior-driven features. These results also validate the inclusion of features with low standalone correlation, as they contribute through interaction effects along the manifold and would be lost under naive pruning.

While feature group ablation is a widely used method for assessing the necessity of input variables, its use in this study was constrained by the scale and nature of our experimental setup. With 84 CVRP instances, twelve supervised learning models, and millions of feature rows extracted from parallel GVNS executions, applying ablation would require retraining hundreds of models across altered feature sets and recomputing search-based evaluations. Instead, we adopted a computationally efficient and theoretically supported strategy based on intrinsic dimensionality estimates and multi-method attribution. Recent studies demonstrate that combining dimensionality reduction with grouped or aggregated attribution methods, such as groupShapley, PCA-based explanations, and hybrid feature scoring, can offer scalable and reliable insights into feature relevance [90–92]. In our experiments, this approach revealed that predictive variance concentrates within a low-dimensional subspace (Table 7), supporting our focus on contextual interactions rather than isolated input removal. This methodological trade-off prioritizes interpretability and resource feasibility without compromising the depth of feature relevance analysis.

As shown in Section 3.6, LightGBM captured interactions between structural and temporal features, while ensemble methods leaned toward behavioral patterns. LightGBM achieved the highest scores (0.96 recall, 0.94 PR-AUC), reflecting its strength in identifying operators that yield significant improvements, likely due to its robustness to class imbalance and its capacity to capture feature interactions.

TRACE-VNS consistently outperformed standard GVNS with statistically significant gains. Average improvements exceeded 30 percent and reached 80 percent in some cases, confirming the advantage of context-aware operator selection. Though developed within GVNS, TRACE's modular structure, which separates data extraction, inference, and heuristic logic, enables direct integration with other metaheuristics, as demonstrated in Appendix C. This complements recent work by [34], who applied offline classifiers at the local search stage within GRASP to guide initial solution selection.

While TRACE-VNS generalizes well across most settings, the observed drop in X-Set performance suggests sensitivity to distributional shifts in instance structure or search dynamics. Such generalization challenges are common in neural combinatorial optimization models, particularly for routing problems like CVRP, where shifts in graph topology or scale can degrade performance [93,94]. Although no adaptation strategies were applied in this study, future extensions could explore lightweight solutions such as instance reweighting, subspace alignment, or fine-tuning on structurally similar graphs. Techniques like multi-view graph contrastive learning [94], scale-adaptive policy tuning (Wang et al. 2023), and cross-problem transfer learning [93] demonstrate promising pathways for addressing these issues. TRACE's modular design supports such methods without altering the metaheuristic core, enabling targeted adaptation where transferability is critical. Empirically validating such adaptation strategies remains an important direction for future work to improve generalization in unseen or structurally divergent scenarios.

In Simulated Annealing, TRACE can estimate the success probability of a candidate move, influencing acceptance decisions or ranking multiple sampled moves to improve convergence. In Genetic Algorithms, TRACE ranks crossover and mutation variants per generation. Core predictive logic remains unchanged, requiring only minor adaptations to the feature encoding interface. This confirms TRACE's algorithm-agnostic design and extendibility beyond GVNS, particularly in contexts where modular integration is essential. Several parallel efforts across the literature, including ensemble-based meta-models, hybrid optimization frameworks, and classifier-guided heuristics [95–99], further reinforce the potential for such architecture-agnostic enhancements.

6. Conclusion

This study introduces TRACE, a modular supervised learning framework that guides neighborhood selection in metaheuristics using features extracted from execution traces. Applied to GVNS for the CVRP, TRACE shows that offline-trained models, especially tree ensembles, can predict operator effectiveness accurately with negligible runtime cost.

Results indicate that learning from action outcomes, temporal dynamics, and graph structure enables informed search control, outperforming fixed or random strategies. LightGBM achieved the highest PR-AUC (0.94), with Random Forest and Extra Trees also performing strongly, confirming model robustness.

Feature analysis revealed that action-reward history and search state consistently drive accuracy, while graph metrics add contextual value. Dimensionality analysis showed high feature redundancy, supporting compact, interpretable models without loss in performance.

Beyond 30% gains in solution quality, Trace illustrates how supervised ML can modularize and generalize operator selection. Its algorithm-agnostic design supports transfer to other metaheuristics, enabling adaptive and explainable optimization under deployment constraints.

7. Future work

Although the current framework relies on off-line-trained models that remain fixed during execution, the architecture accommodates extensions involving dynamic retraining. This supports reproducibility and modularity, but limits adaptivity. Tree-based models, such as LightGBM and Random Forest, perform well but do not inherently support incremental updates once deployed. Although online adaptation is technically feasible with alternative models, such as neural networks or online gradient descent methods, these introduce additional computational overhead and stability concerns, particularly in edge or

resource-constrained settings. The decision to rely on offline training reflects a conscious trade-off between robustness and adaptability. Future work may explore lightweight online learning, ensemble-based adaptation, or periodic partial retraining to improve responsiveness without sacrificing stability.

Integrating Graph Neural Networks (GNNs) could further enhance the expressiveness of graph-based features and improve generalization across structurally complex instances. Hybrid approaches that combine supervised pretraining with reinforcement learning may enable more adaptive decision making under dynamic or previously unseen problem conditions. Moreover, extending TRACE's granularity to include coarse-grained decisions, such as the binary local search classification explored by [34], may open complementary pathways for improving search efficiency across metaheuristic families. For rolling-horizon or reactive scheduling problems, time-series architectures, such as NHITS [100], which outperform legacy models like N-BEATS, could offer additional value, although their impact would depend on specific domain characteristics [101,102]. Kolmogorov–Arnold Networks (KANs), and especially their reversible mixture extensions [103], offer a promising pathway for interpretable learning of feature dependencies in complex decision landscapes.

Finally, future pipelines could collect runtime data to monitor search dynamics and detect performance drift, triggering periodic retraining or feature reassessment to support automated updates and long-term adaptability. Automated drift detection and re-calibration would reinforce the long-term sustainability of ML-driven metaheuristics in dynamic environments.

CRediT authorship contribution statement

Panagiotis Kalatzantonakis: Writing – review & editing, Writing – original draft, Visualization, Software. **Angelo Sifaleras:** Writing – review & editing, Validation, Supervision, Methodology. **Nikolaos Samaras:** Writing – review & editing, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. CVRP mathematical formulation

The Two-Index Vehicle Flow formulation of the CVRP utilizes an undirected weighted graph $G = (V, E)$ where $V = \{0, 1, \dots, n\}$ includes $n + 1$ nodes, with node 0 designated as the depot and nodes $V' = \{1, \dots, n\}$ representing customers. Each edge (i, j) in E has an associated non-negative cost c_{ij}, and a binary variable e_{ij} indicating if edge (i, j) is traversed. Each customer $i \in V'$ has a demand d_i, and there are K identical vehicles each with capacity C located in the depot.

The objective is to minimize the total travel cost:

$$\min \sum_{i=0}^{n} \sum_{j=0}^{n} c_{ij} e_{ij} \tag{A.1a}$$

Subject to the following constraints:

$$\sum_{\substack{i=0 \\ i \neq j}}^{n} e_{ij} = 1 \qquad \forall j \in V' \tag{A.2a}$$

$$\sum_{\substack{j=0 \\ i \neq j}}^{n} e_{ji} = 1 \qquad \forall i \in V' \tag{A.2b}$$

$$\sum_{i=0}^{n} e_{i0} = K \tag{A.2c}$$

$$\sum_{j=0}^{n} e_{0j} = K \tag{A.2d}$$

$$u_i - u_j + C e_{ij} \leq C - d_j \qquad \forall i, j \in V', i \neq j, \text{ where } d_i + d_j \leq C \tag{A.2e}$$

$$d_i \leq u_i \leq C \qquad \forall i \in V' \tag{A.2f}$$

$$e_{ij} \in \{0, 1\} \qquad \forall i, j \in V \tag{A.2g}$$

Constraints (A.2a)–(A.2b) ensure each customer is visited exactly once, fulfilling demands. Constraints (A.2c)–(A.2d) enforce that K vehicles leave and return to the depot. Constraints (A.2e)–(A.2f) eliminate sub-tours and enforce capacity limits to ensure feasible routes.

Appendix B. TRACE-VNS integration: Additional notes

B.1. VNS

VNS alternates between shaking (diversification) and local search (intensification), the latter using either basic descent or Variable Neighborhood Descent (VND) [104], which explores increasingly complex neighborhoods via random, deterministic, or hybrid strategies. Embedding VND in VNS yields *General VNS* (GVNS) [105], which improves performance by sequencing shaking and VND to iteratively improve a solution x until a termination condition is met. Parallel GVNS evaluates multiple neighborhoods simultaneously, accelerating convergence and reducing runtime. [14,15,106,107].

Algorithm 1: General Variable Neighborhood Search

Input : x: current solution, s_{max}: maximum number of steps, k_{max}: number of neighborhoods, t_{max}: maximum execution time
Output: Optimized solution x

$step \leftarrow 0$;
while $step < s_{max}$ *and CpuTime()* $< t_{max}$ **do**
  $x' \leftarrow$ **Shake**(x, k_{max});
  $x'' \leftarrow$ **VND**(x', k_{max});
  $x \leftarrow x''$;
  $step \leftarrow step + 1$;
end
return x;

Algorithm 2: Parallel Variable Neighborhood Descent (Parallel VND)

Input : x: initial solution, k_{max}: number of neighborhoods
Output: Optimized solution x

$l \leftarrow 1$;
while $l \leq k_{max}$ **do**
  $x' \leftarrow$ Best found from parallel exploration of $\mathcal{N}_l(x)$;
  if $f(x') < f(x)$ **then**
    $x \leftarrow x'$;
    $l \leftarrow 1$;
  else
    $l \leftarrow l + 1$;
  end
end
return x;

B.2. Feature formulation and contextualization

B.2.1. Graph metrics

Graph metrics. Let R be the set of all routes, N the total number of customers, D the depot location, r an individual route within R, G_r the center of gravity of route r, n_i^r customer i in route r, $d(a, b)$ the Euclidean distance between points a and b, L_{G_r} the projection line through G_r, and $I(r_1, r_2)$ the number of geometric intersections between routes r_1 and r_2.

$\frac{\sum_{i=1}^{\lvert R\rvert-1}\sum_{j=i+1}^{\lvert R\rvert} I\left(r_i,r_j\right)}{N}$	***Average number of intersections per client.*** *Captures the degree of overlap between routes, normalized by the total number of customers.*
$\frac{\sum_{r\in R}\max_{i\in\{1,\ldots,\lvert r\rvert-1\}} d\left(n_i^r,n_{i+1}^r\right)}{\lvert R\rvert}$	***Longest distance between two connected customers, per route.*** *Measures the longest direct connection between two consecutive clients within a route.*
$\sum_{r\in R}\left(d\left(D,n_1^r\right)+d\left(n_{\lvert r\rvert}^r,D\right)\right)$	***Sum of distances between depot and directly-connected customers.*** *Quantifies the direct connectivity between the depot and customer endpoints.*
$\frac{\sum_{r_1\in R}\sum_{r_2\in R\setminus r_1} d\left(G_{r_1},G_{r_2}\right)}{\lvert R\rvert\cdot(\lvert R\rvert-1)}$	***Average distance between routes (their centers of gravity).*** *Captures spatial separation between routes based on their centroids.*
$\sum_{r\in R}\left(\max_i d\left(L_{G_r},n_i\right)-\min_i d\left(L_{G_r},n_i\right)\right)$	***Average width per route.*** *Represents the lateral spread of customer coordinates in each route.*
$\frac{\sum_{r\in R}\max_{i,j}\mathrm{rad}\left(n_i^r,n_j^r\right)}{\lvert R\rvert}$	***Average span in radian per route.*** *Measures the angular spread of customers in polar coordinates.*
$\frac{\sum_{r\in R}\sum_{i=1}^{\lvert r\rvert}\left(d\left(L_{G_r},n_i\right)\right)^+}{N}$	***Average compactness per route, measured by width.*** *Evaluates route cohesion based on width.*
$\frac{\sum_{r\in R}\sum_{i=1}^{\lvert r\rvert}\mathrm{rad}\left(G_r,n_i\right)}{N}$	***Average compactness per route, measured by radian.*** *Captures compactness in terms of angular spread.*
$\frac{\sum_{r\in R}\max_i d\left(n_i^r,D\right)}{\lvert R\rvert}$	***Average depth per route.*** *Represents the farthest customer from the depot in each route.*
$\sqrt{\frac{\sum_{r\in R}\left(\lvert r\rvert-\frac{N}{\lvert R\rvert}\right)^2}{\lvert R\rvert}}$	***Standard deviation of the number of customers per route.*** *Measures route load imbalance.*
$H_{\mathrm{isc}}=-\sum_{r_1<r_2} p_{r_1,r_2}\log p_{r_1,r_2}$ $p_{r_1,r_2}=I(r_1,r_2)/\sum I$	***Intersection-Based Entropy.*** *Measures how evenly intersections are distributed across routes.*

B.2.2. Upper confidence bound

$\mathrm{UCB}(a)=Q(a)+c\cdot\sqrt{\frac{\ln t}{N(a)}}$ where a is the action, $Q(a)$ its estimated reward, $N(a)$ its selection count, t the current time step, and c the exploration–exploitation parameter.

Algorithm 3: UCB Calculation for Action Selection in GVNS

Input : `action_rewards`: rewards per action, `action_counts`: times each action was selected, `total_count`: total actions selected, C: exploration coefficient, `tie_breaker`: tie resolution function
Output: `ucb_values`: UCB values per action, `ucb_choice`: Action with the highest UCB value

```
function calculate_ucb_scores(rewards, counts, total_count, C, tie_breaker)
    ucb_values ← {};
    foreach action a in rewards do
        if counts[a] > 0 then
            avg_reward ← rewards[a] / counts[a] ; // Mean reward for action a
            exploration_term ← C · sqrt(ln(total_count) / counts[a]);
            // Encourage exploration of infrequent actions
        end
        else
            avg_reward ← 0,  exploration_term ← ∞ ;
            // Prioritize untried actions
        end
        ucb_values[a] ← round(avg_reward + exploration_term, 3);
    end
    ucb_choice ← arg max(ucb_values) ; // Select action with highest UCB
    if multiple max values then
        ucb_choice ← tie_breaker(counts) ; // Resolve ties if needed
    end
    return ucb_values, ucb_choice ; // Return UCB scores and chosen action
```

B.3. Model hyperparameters

See Table B.18.

B.4. Feature notation reference

For clarity, Table B.19 summarizes the key feature notations and abbreviations used in TRACE-VNS, excluding graph metrics, which are already explained in Appendix B.2.1.

Appendix C. Extending TRACE to alternative metaheuristics

The following pseudocode illustrates how the TRACE process can be structurally integrated into metaheuristics beyond GVNS. Each integration preserves the core logic of TRACE: a pretrained model $\mathcal{M}$ ranks candidate actions or moves using context features extracted from the current search state. These abstractions are not full implementations but offer drop-in templates to enhance operator or move decisions in Simulated Annealing and Genetic Algorithms.

Offline data collection and model training occur prior to execution. The model $\mathcal{M}$ is trained using labeled data derived from prior runs, where each feature vector represents a local decision context and the label indicates whether a move led to improvement.

Algorithm 4: TRACE Phase 1: Data Collection and Offline Model Training (Framework-Agnostic)

Input : Heuristic controller $\mathcal{H}$, operator set $\mathcal{N}$
Output: Trained model $\mathcal{M}$

```
D ← ∅ ; // Initialize empty dataset
foreach instance I do
    x ← InitialSolution(I);
    while not terminated do
        op ← SelectRandom(N);
        x' ← Apply(x, op);
        f ← ExtractFeatures(x, x', op);
        y ← 𝕀[Improved(x, x')];
        D ← D ∪ {(f, y)};
        x ← x' ; // Apply regardless of success (unbiased)
    end
end
M ← TrainBinaryClassifier(D) ; // Label: 1 = improvement, 0 = no improvement
return M;
```

C.1. Integrating TRACE into simulated annealing: Move-level prediction

In Simulated Annealing, TRACE evaluates each candidate move before applying the Metropolis acceptance rule. The prediction augments probabilistic acceptance, enabling earlier rejection of poor candidates and accelerating convergence without modifying the core temperature schedule.

Table C.20
Guidelines for integrating TRACE into Simulated Annealing (SA).

Role and integration targets	
Integration scope	TRACE scores sampled moves based on contextual features to guide acceptance
Model objective	Predicts the likelihood that a candidate move improves the solution
Prediction cycle and inference	
Inference timing	Per iteration, TRACE scores all sampled moves before acceptance
Inference cost	Lightweight; adds < 0.5 ms latency per iteration
Acceptance rule modification	
Baseline strategy	Classical Metropolis rule: accept if $\Delta f < 0$ or with probability $e^{-\Delta f/T}$
TRACE integration	Metropolis rule is retained; TRACE score is used to prioritize or filter proposed moves
Contextual input features	
Current objective	Cost of the current solution $f(x)$
Candidate objective	Cost of the candidate solution $f(x')$
Objective delta Δf	Improvement magnitude: $f(x') - f(x)$
Temperature T	Encodes search phase; used as a feature
Search phase and move state	Includes iteration index and local move details (e.g., arc length delta, node distance)
Graph-level descriptors	Route structure: length, dispersion, and topological features
Search history stats	Rolling indicators: acceptance ratio, last improvement step, or entropy

Algorithm 5: TRACE-Augmented Simulated Annealing (No Operator Set)

Input : x: initial solution, T_0: initial temperature, T_{min}: stopping temperature, r: cooling rate, $\mathcal{M}$: pretrained model
Output: Optimized solution x

$T \leftarrow T_0$;
while $T > T_{min}$ **do**
  $x' \leftarrow$ **SampleNeighbor**(x) ; // Standard neighborhood move
  $f \leftarrow$ **ExtractFeatures**(x, x', T) ; // Features include temperature and move context
  $p \leftarrow \mathcal{M}.predict(f)$; // Predicted likelihood of improvement
  $\Delta f \leftarrow f(x') - f(x)$;
  if $\Delta f < 0$ ***or*** $(p > \tau \wedge \exp(-\Delta f/T) >$ ***rand***$(0,1))$ **then**
    $x \leftarrow x'$; // Accept based on model prediction and Metropolis condition
  end
  $T \leftarrow T \cdot r$;
end
return x;

C.2. Integrating TRACE into genetic algorithms: Operator ranking per generation

In Genetic Algorithms, TRACE ranks candidate offspring by evaluating each mutation or crossover operator. The most promising operator is selected per parent pair using the model's prediction, ensuring efficient use of genetic diversity without exhaustive evaluation.

Algorithm 6: TRACE-Guided Genetic Algorithm

Input : P: population, G: generations, $\mathcal{N}$: mutation/crossover ops, $\mathcal{M}$: pretrained model
Output: Final population P

for $g \leftarrow 1$ **to** G **do**
  $P_{new} \leftarrow \emptyset$;
  while $|P_{new}| < |P|$ **do**
    $(p_1, p_2) \leftarrow$ **SelectParents**(P);
    foreach $op \in \mathcal{N}$ **do**
      $child_{op} \leftarrow$ **Apply**(p_1, p_2, op);
      $f_{op} \leftarrow$ **ExtractFeatures**$(p_1, p_2, child_{op}, op, g)$; // operator and generation info
      $p_{op} \leftarrow \mathcal{M}.predict(f_{op})$;
    end
    $op^* \leftarrow \arg\max_{op} p_{op}$;
    $child \leftarrow$ **Apply**(p_1, p_2, op^*);
    $P_{new} \leftarrow P_{new} \cup \{child\}$;
  end
  $P \leftarrow$ **SurvivorSelection**$(P \cup P_{new})$;
end
return P;

Data availability

We provide a link to a widely-used benchmark dataset.

References

[1] G.B. Dantzig, J.H. Ramser, The truck dispatching problem, Manag. Sci. 6 (1) (1959) 80–91.
[2] G. Laporte, Y. Nobert, M. Desrochers, Optimal routing under capacity and distance restrictions, Oper. Res. 33 (5) (1985) 1050–1073.
[3] R. Baldacci, E. Hadjiconstantinou, A. Mingozzi, An exact algorithm for the Capacitated Vehicle Routing Problem based on a two-commodity network flow formulation, Oper. Res. 52 (5) (2004) 723–738.
[4] J.P.Q. dos Santos, J.D. de Melo, A.D.D. Neto, D. Aloise, Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search, Expert Syst. Appl. 41 (10) (2014) 4939–4949.
[5] E. Jabir, V.V. Panicker, R. Sridharan, Design and development of a hybrid ant colony-variable neighbourhood search algorithm for a multi-depot green vehicle routing problem, Transp. Res. Part D: Transp. Environ. 57 (2017) 422–457.
[6] B. Chen, R. Qu, R. Bai, W. Laesanklang, A variable neighborhood search algorithm with reinforcement learning for a real-life periodic vehicle routing problem with time windows and open routes, RAIRO- Oper. Res. 54 (5) (2020) 1467–1494.
[7] L.D.P. Pugliese, D. Ferone, P. Festa, F. Guerriero, G. Macrina, Combining variable neighborhood search and machine learning to solve the vehicle routing problem with crowd-shipping, Optim. Lett. (2022) 1–23.
[8] P. Kalatzantonakis, A. Sifaleras, N. Samaras, A reinforcement learning-variable neighborhood search method for the Capacitated Vehicle Routing Problem, Expert Syst. Appl. 213 (2023) 118812.
[9] N. Mladenović, P. Hansen, Variable neighborhood search, Comput. Oper. Res. 24 (11) (1997) 1097–1100.
[10] R. Elshaer, H. Awad, A taxonomic review of metaheuristic algorithms for solving the vehicle routing problem and its variants, Comput. Ind. Eng. 140 (2020) 106242.
[11] M. Polacek, S. Benkner, K.F. Doerner, R.F. Hartl, A cooperative and adaptive variable neighborhood search for the multi depot vehicle routing problem with time windows, Bus. Res. 1 (2008) 207–218.
[12] O. Polat, A parallel variable neighborhood search for the vehicle routing problem with divisible deliveries and pickups, Comput. Oper. Res. 85 (2017) 71–86.
[13] R. Todosijević, S. Hanafi, D. Urošević, B. Jarboui, B. Gendron, A general variable neighborhood search for the swap-body vehicle routing problem, Comput. Oper. Res. 78 (2017) 468–479.
[14] P. Kalatzantonakis, A. Sifaleras, N. Samaras, Cooperative versus non-cooperative parallel variable neighborhood search strategies: a case study on the Capacitated Vehicle Routing Problem, J. Global Optim. 78 (2) (2020) 327–348.
[15] P. Kalatzantonakis, A. Sifaleras, N. Samaras, On a cooperative VNS parallelization strategy for the Capacitated Vehicle Routing Problem, in: Learning and Intelligent Optimization: 13th International Conference, LION 13, Chania, Crete, Greece, May 27–31, 2019, Revised Selected Papers 13, Springer, 2019, pp. 231–239.

[16] P. Hansen, N. Mladenović, Variable neighborhood search: Principles and applications, European J. Oper. Res. 130 (3) (2001) 449–467.

[17] P. Hansen, N. Mladenović, J. Brimberg, J.A.M. Pérez, Variable neighborhood search, in: Handbook of Metaheuristics, Springer, 2019, pp. 57–97.

[18] F. Parreño, R. Alvarez-Valdés, J.F. Oliveira, J.M. Tamarit, Neighborhood structures for the container loading problem: a VNS implementation, J. Heuristics 16 (2010) 1–22.

[19] G. Moslehi, D. Khorasanian, A hybrid variable neighborhood search algorithm for solving the limited-buffer permutation flow shop scheduling problem with the makespan criterion, Comput. Oper. Res. 52 (2014) 260–268.

[20] M.R. Khouadjia, B. Sarasola, E. Alba, L. Jourdan, E.-G. Talbi, A comparative study between dynamic adapted PSO and VNS for the vehicle routing problem with dynamic requests, Appl. Soft Comput. 12 (4) (2012) 1426–1439.

[21] K. Li, H. Tian, A multiobjective variable neighborhood search with learning and swarm for permutation flowshop scheduling with sequence-dependent setup times, Processes 10 (9) (2022) 1786.

[22] S. Belhaiza, R. MḤallah, G.B. Brahim, A new hybrid genetic variable neighborhood search heuristic for the vehicle routing problem with multiple time windows, in: 2017 IEEE Congress on Evolutionary Computation, CEC 2017, IEEE, 2017, pp. 1319–1326.

[23] C.-C. Lin, J.-R. Kang, Y.-L. Liang, C.-C. Kuo, Simultaneous feature and instance selection in big noisy data using memetic variable neighborhood search, Appl. Soft Comput. 112 (2021) 107855.

[24] D. Kovačević, N. Mladenović, B. Petrović, P. Milošević, DE-VNS: Self-adaptive differential evolution with crossover neighborhood search for continuous global optimization, Comput. Oper. Res. 52 (2014) 157–169.

[25] R.A. Aziz, M. Ayob, Z. Othman, The effect of learning mechanism in variables neighborhood search, in: 2012 4th Conference on Data Mining and Optimization, DMO 2012, IEEE, 2012, pp. 109–113.

[26] S. Su, D. Xiong, H. Yu, X. Dong, A multiple leaders particle swarm optimization algorithm with variable neighborhood search for multiobjective fixed crowd carpooling problem, Swarm Evol. Comput. 72 (2022) 101103.

[27] W. Sun, Z. Luo, X. Hu, W. Pedrycz, J. Shi, An improved variable neighborhood search algorithm embedded temporal and spatial synchronization for vehicle and drone cooperative routing problem with pre-reconnaissance, Swarm Evol. Comput. 91 (2024) 101699.

[28] A. Stenger, D. Vigo, S. Enz, M. Schwind, An adaptive variable neighborhood search algorithm for a vehicle routing problem arising in small package shipping, Transp. Sci. 47 (1) (2013) 64–80.

[29] E.K. Burke, J. Li, R. Qu, A hybrid model of integer programming and variable neighbourhood search for highly-constrained nurse rostering problems, European J. Oper. Res. 203 (2) (2010) 484–493.

[30] U.R. Teixeira, M.J.F. Souza, S.R. de Souza, V.N. Coelho, An adaptive VNS and skewed GVNS approaches for school timetabling problems, in: Variable Neighborhood Search: 6th International Conference, ICVNS 2018, Sithonia, Greece, October 4–7, 2018, Revised Selected Papers 6, Springer, 2019, pp. 101–113.

[31] S. Loudni, P. Boizumault, N. Levasseur, Advanced generic neighborhood heuristics for VNS, Eng. Appl. Artif. Intell. 23 (5) (2010) 736–764.

[32] S. Thevenin, N. Zufferey, Learning variable neighborhood search for a scheduling problem with time windows and rejections, Discrete Appl. Math. 261 (2019) 344–353.

[33] P. Karakostas, A. Sifaleras, Learning-assisted improvements in adaptive variable neighborhood search, Swarm Evol. Comput. 94 (2025) 101887.

[34] J.P. Mesa, A. Montoya, R. Ramos-Pollan, M. Toro, Machine-learning component for multi-start metaheuristics to solve the Capacitated Vehicle Routing Problem, Appl. Soft Comput. 173 (2025) 112916.

[35] Z. Wang, R. Bai, F. Khan, E. Ozcan, T. Zhang, GASE: Graph attention sampling with edges fusion for solving vehicle routing problems, 2024, arXiv preprint arXiv:2405.12475.

[36] N.D. Kullman, N. Dudorov, M. Cousineau, J.E. Mendoza, J.C. Goodson, Gamifying the vehicle routing problem with stochastic requests, 2024, URL https://arxiv.org/abs/1911.05922.

[37] F.D. Hildebrandt, B.W. Thomas, M.W. Ulmer, Opportunities for reinforcement learning in stochastic dynamic vehicle routing, Comput. Oper. Res. 150 (2023) 106071.

[38] T. Guo, Y. Mei, W. Du, Y. Lv, Y. Li, T. Song, Emergency scheduling of aerial vehicles via graph neural neighborhood search, IEEE Trans. Artif. Intell. (2025).

[39] M. Gasse, S. Bowly, Q. Cappart, J. Charfreitag, L. Charlin, D. Chételat, A. Chmiela, J. Dumouchelle, A. Gleixner, A.M. Kazachkov, et al., The machine learning for combinatorial optimization competition (ml4co): Results and insights, in: NeurIPS 2021 Competitions and Demonstrations Track, PMLR, 2022, pp. 220–231.

[40] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: a methodological tour d'horizon, European J. Oper. Res. 290 (2) (2021) 405–421.

[41] M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A.M. Karimi-Mamaghan, E.-G. Talbi, Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art, European J. Oper. Res. 296 (2) (2022) 393–422.

[42] F. Lucas, R. Billot, M. Sevaux, K. Sörensen, Reducing space search in combinatorial optimization using machine learning tools, in: Learning and Intelligent Optimization: 14th International Conference, LION 14, Athens, Greece, May 24–28, 2020, Revised Selected Papers 14, Springer, 2020, pp. 143–150.

[43] T. Guo, C. Han, S. Tang, M. Ding, Solving combinatorial problems with machine learning methods, Nonlinear Comb. Optim. (2019) 207–229.

[44] F. Arnold, K. Sörensen, What makes a VRP solution good? The generation of problem-specific knowledge for heuristics, Comput. Oper. Res. 106 (2019) 280–288.

[45] H. Wang, Y. Jin, A random forest-assisted evolutionary algorithm for data-driven constrained multiobjective combinatorial optimization of trauma systems, IEEE Trans. Cybern. 50 (2) (2018) 536–549.

[46] W.B. Yates, Offline Learning for Sequence-Based Selection Hyper-Heuristics, University of Exeter (United Kingdom), 2020.

[47] F. Xue, G.Q. Shen, Design of an efficient hyper-heuristic algorithm CMA-VNS for combinatorial black-box optimization problems, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2017, pp. 1157–1162.

[48] M. Sarhani, S. Voß, Adaptive neighborhood search based on landscape learning: A TSP study, in: Evolutionary Computation in Combinatorial Optimization: 25th European Conference, EvoCOP 2025, Held As Part of EvoStar 2025, Trieste, Italy, April 23–25, 2025, Proceedings, Springer Nature, 2025, p. 200.

[49] L.D. Pugliese, D. Ferone, P. Festa, F. Guerriero, G. Macrina, Combining variable neighborhood search and machine learning to solve the vehicle routing problem with crowd-shipping, Optim. Lett. (2023) 1–23.

[50] N. Čolić, P. Milošević, I. Dragović, M.S. Ćeranić, IBA-VNS: A logic-based machine learning algorithm and its application in surgery, Mathematics 12 (7) (2024) 950.

[51] G. Ghiani, G. Laporte, E. Manni, Model-based automatic neighborhood design by unsupervised learning, Comput. Oper. Res. 54 (2015) 108–116.

[52] D. Pambudi, M. Kawamura, Constructing the neighborhood structure of VNS based on binomial distribution for solving QUBO problems, Algorithms 15 (6) (2022) 192.

[53] B. Radomirovic, N. Bacanin, L. Jovanovic, V. Simic, A. Njegus, D. Pamucar, M. Köppen, M. Zivkovic, Optimizing long-short term memory neural networks for electroencephalogram anomaly detection using variable neighborhood search with dynamic strategy change, Complex Intell. Syst. 10 (6) (2024) 7987–8009.

[54] A. Corrêa, C. Silva, L. Xu, A. Brintrup, S. Moniz, TuneNSearch: a hybrid transfer learning and local search approach for solving vehicle routing problems, 2025, arXiv preprint arXiv:2503.12662.

[55] CVRPLIB - all instances, 2021, http://vrp.atd-lab.inf.puc-rio.br/index.php/en, (Accessed 22 June 2025).

[56] P. Augerat, D. Naddef, J. Belenguer, E. Benavent, A. Corberan, G. Rinaldi, Computational results with a branch and cut code for the capacitated vehicle routing problem, 1995.

[57] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, A. Subramanian, New benchmark instances for the Capacitated Vehicle Routing Problem, European J. Oper. Res. 257 (3) (2017) 845–858.

[58] N. Christofides, S. Eilon, An algorithm for the vehicle-dispatching problem, J. Oper. Res. Soc. 20 (3) (1969) 309–318.

[59] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine Learning in Python, J. Mach. Learn. Res. 12 (2011) 2825–2830.

[60] R Core Team, R: A Language and Environment for Statistical Computing, R Foundation for Statistical Computing, Vienna, Austria, 2025, URL https://www.R-project.org.

[61] RStudio Team, RStudio: Integrated Development Environment for R, RStudio, PBC., Boston, MA, 2025, URL http://www.rstudio.com.

[62] J. Bac, E.M. Mirkes, A.N. Gorban, I. Tyukin, A. Zinovyev, Scikit-dimension: a python package for intrinsic dimension estimation, Entropy 23 (10) (2021) 1368.

[63] R.L. Thorndike, Who belongs in the family? Psychometrika 18 (4) (1953) 267–276.

[64] K. Berahmand, F. Daneshfar, E.S. Salehi, Y. Li, Y. Xu, Autoencoders and their applications in machine learning: a survey, Artif. Intell. Rev. 57 (2) (2024) 28.

[65] J.R. Quinlan, Induction of decision trees, Mach. Learn. 1 (1) (1986) 81–106.

[66] J.R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, 1993.

[67] L. Breiman, Random forests, Mach. Learn. 45 (2001) 5–32.

[68] A. Cutler, D.R. Cutler, J.R. Stevens, Random forests, Ensemble Mach. Learn.: Methods Appl. (2012) 157–175.

[69] G. Biau, Analysis of a random forests model, J. Mach. Learn. Res. 13 (1) (2012) 1063–1095.

[70] P. Geurts, D. Ernst, L. Wehenkel, Extremely randomized trees, Mach. Learn. 63 (1) (2006) 3–42.

[71] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[72] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, Adv. Neural Inf. Process. Syst. 30 (2017).

[73] L. Prokhorenkova, G. Gusev, A. Vorobev, A.V. Dorogush, A. Gulin, CatBoost: unbiased boosting with categorical features, Adv. Neural Inf. Process. Syst. 31 (2018).
[74] Y. Freund, R.E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting, J. Comput. System Sci. 55 (1) (1997) 119–139.
[75] J.R. Quinlan, Experiments with a new boosting algorithm, in: Machine Learning: Proceedings of the 13th International Conference, Vol. 96, 1996, pp. 148–156.
[76] N.S. Altman, An introduction to kernel and nearest-neighbor nonparametric regression, Amer. Statist. 46 (3) (1992) 175–185.
[77] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536.
[78] J.H. Friedman, Greedy function approximation: a gradient boosting machine, Ann. Stat. (2001) 1189–1232.
[79] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.
[80] C. Strobl, A.-L. Boulesteix, A. Zeileis, T. Hothorn, Bias in random forest variable importance measures: Illustrations, sources and a solution, BMC Bioinformatics 8 (1) (2007) 25.
[81] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. 30 (2017).
[82] A. Saabas, Interpreting random forests, 2015, URL http://blog.datadive.net/interpreting-random-forests.
[83] V. Bewick, L. Cheek, J. Ball, Statistics review 10: further nonparametric methods, Crit. Care 8 (2004) 1–4.
[84] B. Rosner, R.J. Glynn, M.-L.T. Lee, The Wilcoxon signed rank test for paired comparisons of clustered data, Biometrics 62 (1) (2006) 185–192.
[85] D. Mohapatra, S. Singh, R. Pradhan, S.R. Behera, Integrating structural features for link prediction in directed social network using supervised machine learning with SHapley Additive exPlanations, in: 2023 OITS International Conference on Information Technology, OCIT, IEEE, 2023, pp. 468–472.
[86] N. Bui, H.T. Nguyen, V.A. Nguyen, R. Ying, Explaining graph neural networks via structure-aware interaction index, 2024, arXiv preprint arXiv:2405.14352.
[87] X. Xian, J. Liu, T. Wu, L. Yuan, C. Wang, B. Chen, GraphGI: A GNN explanation method using game interaction, 2024, arXiv preprint arXiv:2409.15698.
[88] H. Yuan, H. Yu, J. Wang, K. Li, S. Ji, On explainability of graph neural networks via subgraph explorations, in: International Conference on Machine Learning, PMLR, 2021, pp. 12241–12252.
[89] A. Perotti, P. Bajardi, F. Bonchi, A. Panisson, Graphshap: Motif-based explanations for black-box graph classifiers, 2022, arXiv preprint arXiv:2202.08815.
[90] Q. Au, J. Herbinger, C. Stachl, B. Bischl, G. Casalicchio, Grouped feature importance and combined features effect plot, Data Min. Knowl. Discov. 36 (4) (2022) 1401–1450.
[91] D. Rengasamy, B.C. Rothwell, G.P. Figueredo, Towards a more reliable interpretation of machine learning outputs for safety-critical systems using feature importance fusion, Appl. Sci. 11 (24) (2021) 11854.
[92] H. Deng, N. Zou, M. Du, W. Chen, G. Feng, X. Hu, A unified taylor framework for revisiting attribution methods, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 11462–11469.
[93] A. Yaddaden, S. Harispe, M. Vasquez, Is transfer learning helpful for neural combinatorial optimization applied to vehicle routing problems? Comput. Inform. 41 (1) (2022) 172–190.
[94] Y. Jiang, Z. Cao, Y. Wu, J. Zhang, Multi-view graph contrastive learning for solving vehicle routing problems, in: Uncertainty in Artificial Intelligence, PMLR, 2023, pp. 984–994.
[95] E.-G. Talbi, Machine learning into metaheuristics: A survey and taxonomy, ACM Comput. Surv. 54 (6) (2021) 1–32.
[96] N. Mehrabi Hashjin, M.H. Amiri, A. Mohammadzadeh, S. Mirjalili, N. Khodadadi, Novel hybrid classifier based on fuzzy type-III decision maker and ensemble deep learning model and improved chaos game optimization, Clust. Comput. 27 (7) (2024) 10197–10234.
[97] M.H. Amiri, M. Pourgholi, N.M. Hashjin, M.K. Ardakani, Monitoring UAV status and detecting insulator faults in transmission lines with a new classifier based on aggregation votes between neural networks by interval type-2 TSK fuzzy system, Soft Comput. 28 (20) (2024) 12141–12174.
[98] A.A. Hussein, E.T. Yaseen, A.N. Rashid, Learnheuristics in routing and scheduling problems: A review, Samarra J. Pure Appl. Sci. 5 (1) (2023) 60–90.
[99] C. Wang, Z. Yu, S. McAleer, T. Yu, Y. Yang, Asp: Learn a universal neural solver!, IEEE Trans. Pattern Anal. Mach. Intell. 46 (6) (2024) 4102–4114.
[100] C. Challu, K.G. Olivares, B.N. Oreshkin, F.G. Ramirez, M.M. Canseco, A. Dubrawski, Nhits: Neural hierarchical interpolation for time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 6989–6997.
[101] V. Cerqueira, L. Roque, C. Soares, Forecasting with deep learning: Beyond average of average of average performance, in: International Conference on Discovery Science, Springer, 2024, pp. 135–149.
[102] P. Kalatzantonakis, A. Sifaleras, N. Samaras, Temporal action analysis in metaheuristics: A machine learning approach, in: Metaheuristics, in: Lecture notes in computer science, Springer Nature Switzerland, Cham, 2024, pp. 365–370.
[103] X. Han, X. Zhang, Y. Wu, Z. Zhang, Z. Wu, KAN4TSF: Are KAN and KAN-based models Effective for Time Series Forecasting? 2024, ArXiv.
[104] A. Duarte, J. Sánchez-Oro, N. Mladenović, R. Todosijević, Variable neighborhood descent, in: R. Martí, P.M. Pardalos, M.G.C. Resende (Eds.), Handbook of Heuristics, Springer International Publishing, Cham, 2018, pp. 341–367.
[105] N. Mladenović, R. Todosijević, D. Urošević, Two level general variable neighborhood search for Attractive traveling salesman problem, Comput. Oper. Res. 52 (2014) 341–348.
[106] J.A. Moreno-Pérez, P. Hansen, N. Mladenovic, Parallel Variable Neighborhood Searchs, Tech. rep., Tech. Rep., Group of Intelligent Computing, Universidad de La Laguna, 2004.
[107] N. Antoniadis, A. Sifaleras, A hybrid CPU-GPU parallelization scheme of variable neighborhood search for inventory optimization problems, Electron. Notes Discrete Math. 58 (2017) 47–54.